



UNIVERSITY STUDENT LEADERS COMPUTERIZED VOTING SYSTEM

SYSTEM DOCUMENTATION

**SUBMITTED BY,
WERE NELLY OTIENO
DSE-01-0085/2023**

**A SYSTEM DOCUMENTATION SUBMITTED IN PARTIAL FULFILMENT FOR THE
AWARD OF DIPLOMA IN SOFTWARE ENGINEERING BY ZETECH UNIVERSITY**

APRIL 2026

DECLARATION

I declare that the contents of this documentation are original and therefore have neither been plagiarized from websites, from other student's work, nor has it been submitted before in a previous semester to Zetech University or any other institution and only contains original work as guided by the unit instructor.

Student Name: _____ Sign: _____ Date: _____

Supervisor Name: _____ Sign: _____ Date: _____

DEDICATION

I hereby dedicate this project to myself for the commitment and perseverance shown throughout its development despite the challenges encountered.

ABSTRACT

This project took three months and is inspired by the need to make student elections more efficient, transparent, and accessible. In many institutions, voting is still done manually through paper ballots and physical queues which often results in delays, errors in vote counting, and lower student participation. To help solve this, I designed and developed an online voting system that allows students to vote easily and securely from anywhere. The platform was built using PHP, MySQL, and basic web technologies like HTML and CSS. It features a secure login system, a simple voting process, and an admin panel where election officials can manage candidates, monitor votes, and view results in real time. The system not only speeds up the election process but also reduces the chances of human error and encourages more students to take part in elections. During testing, users found it easy to use and appreciated the clarity and convenience it offered. Looking ahead, there's potential to add more features like two-factor authentication, email or SMS alerts, and a mobile-friendly version to reach even more users. Overall, this project shows how technology can help improve everyday processes in schools and make student leadership elections smoother and more inclusive.

DEFINITION OF KEY TERMS

Authentication: This is proving a person's identity to ensure only eligible voters listed can vote.

Authorization: Process of determining if the user has permission to access resources or perform specific tasks within the voting system

Database: a central repository for data

Data Backup: It involves production of many copies of data for the purpose of safe-guarding it against data loss.

Data protection: Protecting citizens' private, personal data from unauthorized use, transmission and storage.

Election: A democratic procedure through which one or more persons are chosen as executives or representatives in certain bodies.

Election Integrity: The right to vote in a free, fair and transparent election.

Election Results: Summary of the final outcome of the election.

Encryption: Secures votes transmission and storage, protecting election integrity.

Online Voting: Conducting an online election.

Real-Time Voting: A proposed method of direct democracy where voters represent themselves online.

Remote Voting: Ability to participate in the election from any location using an internet-connected device.

Responsive Design: An interface adapts to the device's layout to enable usability navigation and information looking.

User Interface: A way human interact with an application or website using visual and audio elements.

Voter Anonymity: Ensuring that each vote remain a secret regardless of the voter's choice or voting method.

Voter Eligibility: Determine if the voter is qualified to vote or not.

Voting Session: Period which students are allowed to access the system and cast their votes.

Vote Tallying: Counting and amounting all the votes cast in an election

Voter Turnout: Measures the percentage of voters that have participate in the election (who actually cast a vote).

Webpage: a single page containing particular information of the website e.g. a contact page, built to be accessed from a browser.

ABBREVIATIONS AND ACRONYMS

CSS – Cascaded Style Sheet

DB – Database

ERD – Entity Relationship Diagram

FK – Foreign Key

HTML – Hyper Text Mark Up Language

HTTPS – Hypertext Transfer Protocol Secure

ID – Identification

OTP – One-Time Password

PHP – Hypertext Preprocessor

PK – Primary Key

SMS – Short Message Service

SQL – Structured Query Language

UI – User Interface

URL – Uniform Resource Locator

WI-FI – Wireless Fidelity

LIST OF FIGURES

Fig 2.2.1 Home Page Design

Fig 2.2.2 Signup Page Design

Fig 2.2.3.1 Student Login Page Design

Fig 2.2.3.2 Admin Login Page Design

Fig 2.2.4.1 Student Dashboard Page Design

Fig 2.2.4.2 Admin Dashboard Page Design

Fig 2.3.1 Use Case Diagram

Figure 2.3.2: ERD for the Student Leaders Online Voting System

Figure 2.3.3.1: Voting Process Flowchart

Figure 2.3.3.2: Admin Flowchart

Figure 2.3.4: System Architecture Diagram

Figure 2.4: Class Diagram of the Online Student Voting System

Figure 2.5.1: DFD Level 0

Figure 2.5.2: DFD Level 1

Fig 3.2.1.1: Student Login Interface

Fig 3.2.1.2: Student Login Code

Fig 3.2.1.3: Admin Login Interface

Fig 3.2.1.4: Admin Login Code

Fig 3.2.2: Signup Page

Fig 3.2.3: Signup page code

Fig 3.2.4.1: Admin Interface

Fig 3.2.4.1.1: Create Election Interface

Fig 3.2.4.2: Dashboard Interface Code

Fig 3.2.4.1.2: Manage Candidates Interface

Fig 3.2.4.1.3: Manage Candidates code

Fig 3.2.4.1.4: Manage Students Interface

Fig 3.2.4.1.5: Manage Students Code

Fig 3.2.4.1.6: Monitor Voting Interface

Fig 3.2.4.1.7: Monitor Voting Interface Code

Fig 3.2.4.1.8: View Results Interface

Fig 3.2.5.1: Student Interface

Fig 3.2.5.2: Casting Vote Interface

Fig 3.3.1 Student Registration Code

Fig 3.3.2 Student Validation Login Code

Fig 3.3.3 Student Database

Fig 3.3.4 Admin Validation Login Code

Fig 3.3.5 Admin Database

Fig 3.3.6 Dashboard Validation Code

Fig 3.3.7 Manage Candidate Validation Code

Fig 3.3.8 Monitor Votes Validation Code

Fig 3.3.9 View Results Validation Code

Fig 3.3.10 Publish Results Validation Code

Fig 3.3.11 Election Database

Fig 3.3.12 Vote Validation Code

Fig 3.3.13 Vote Database

LIST OF TABLES

Table 1.4 Functional Requirements

Table 1.5.1 Software Requirements Table

Table 1.5.2 Hardware Requirements

Table 1.6 Project Schedule Breakdown

TABLE OF CONTENTS

Contents

CHAPTER ONE: PROJECT PLANNING AND ANALYSIS	1
Statement of Problem	1
Study Justification.....	1
System Objectives.....	1
General Objective	1
Specific Objectives	1
Functional Requirements.....	2
Breakdown of Tools & Resources to Be Used.....	2
1.5.1 Software Requirements	2
1.5.2 Hardware Requirements.....	3
1.6 Project Schedule Breakdown	1
CHAPTER TWO: DESIGN AND MODELING.....	2
2.1 Introduction to Modelling	2
2.2 User Interface Models	2
2.2.1 Home Page.....	2
2.2.2 Signup Page.....	4
2.2.3 Login Page Design	4
2.2.3.1 Student Login.....	4
2.2.3.2 Admin Login	5
2.2.4 Product Page Design	5
2.2.4.1 Student Dashboard	5
2.2.4.2 Admin Dashboard	6
2.3 Logic Models	7
2.3.1 Use Case Diagrams.....	7
2.3.2 ER Diagrams	8
2.3.3 Flowcharts	9
2.3.4 Architectual Diagrams	11
2.4 Class Diagrams	12
2.5 DFD Diagrams	13
2.5.1 Level 0	13
2.5.2 DFD Level 1	14
CHAPTER THREE: SYSTEM IMPLEMENTATION	15

3.1 Introduction	15
3.2 User Interface Development	15
3.2.1 Login Page	15
3.2.2 Signup page	23
3.2.3 Signup page code	24
3.2.4 Dashboards	26
3.2.5 Student Dashboard	44
3.3 Logic Development	46
3.3.1 Student Registration Validation Code	46
3.3.2 Student Validation Login Code	47
3.3.3 Student Database	48
3.3.4 Admin Validation Login Code	49
3.3.5 Admin Database	50
3.3.6 Dashboard Validation Code	51
3.3.7 Manage Candidate Validation Code	53
3.3.8 Candidate Database	55
3.3.8 Monitor Votes Validation Code	56
3.3.9 View Results Validation Code	57
3.3.10 Publish Results Validation Code	59
3.3.11 Election Database	61
3.3.12 Vote Validation Code	62
3.3.13 Vote Database	63
3.4 Testing	64
3.5 Deployment	65
CHAPTER FOUR: CONCLUSION AND RECOMMENDATION	66
4.1 Conclusion	66
4.2 Recommendation	66
REFERENCES	68

CHAPTER ONE: PROJECT PLANNING AND ANALYSIS

(WORKPLAN)

1.1 Statement of Problem

Inspired by challenges observed in Kenyan Universities such as Taita University, <https://doi.org/10.53555/sshr.v4i4.2794>, students get to elect their leaders after every one academic year, this gives them an opportunity to air out their voices, participate in decision making and create awareness and understanding of the school culture. During these elections, there are low turnouts of students as they physically can't make it to the voting centers. This results to poor decision-making, poor representation of the student body, limited accountability, lack of momentum for campus events and missed opportunities for change. Also the traditional paper-based voting methods are costly and time consuming. The research will focus on how to increase the numbers of voters regardless of their location securely and efficiently and also encourage the students to participate in the school elections.

1.2 Study Justification

The computerized voting system for the school will save time and resources as the traditional voting method involves a lot of paperwork from preparing ballot papers to counting of votes. The traditional voting system leads to printing of paper ballots, hiring people to administer the process and provide security thus increasing costs. The proposed system will help reduce these costs as most of the task force will be implemented online. The proposed system will increase the number of students casting their votes at their convenient time online as there will be no need for physical travel to school. Compared to the traditional paper-based voting system, the computerized voting system will be secure as only authorized users will be able to use it upon authentication making the voting process more transparent. Overall, a computerized system will make the voting process more accessible, secure, cost-effective and efficient to use. This will lead to high turnouts of students participating in the voting process.

1.3 System Objectives

1.3.1 General Objective

- (i)** To develop a student leaders computerized university online voting system.

1.3.2 Specific Objectives

- (i)** To develop a system that ensures each student can vote only once per election.
- (ii)** To develop a system that increases voter turnout by enabling convenient and remote voting.
- (iii)** To design a system that enables real-time monitoring and viewing of election results.
- (iv)** To develop a secured user authentication system.

1.4 Functional Requirements

User	User Activities	Features
Admin	Signup, Login, Monitor Elections and Manage Candidates	Create and manage elections, add/edit/delete candidates, monitor voting progress, view and analyze results, manage user access
Student	Signup, Login, Vote and View Results	Register and log in using admission number, view active elections, select candidates, cast vote, receive vote confirmation, view election results

Table 1.4 Functional Requirements Table

1.5 Breakdown of Tools & Resources to Be Used

1.5.1 Software Requirements

Item	Estimated Cost	Description
OpenSourceDevelopment Tools	Ksh.0.00	Use of free IDEs, frameworks, and libraries e.g., Visual Studio Code, JDK
Security Tools (SSL Certificates, Encryption)	Ksh.0.00	Free SSL/TLS certificates e.g., Let's Encrypt) and open-source encryption libraries
Cloud Hosting (Free Tier)	Ksh.0.00	Use of cloud hosting providers' free tier e.g., GitHub Pages
Database (Free Tier or Open Source)	Ksh.0.00	Use of open-source databases e.g., MySQL PhpMyAdmin
TOTAL	Ksh.0.00	

Table 1.5.1 Software Requirements Table

1.5.2 Hardware Requirements

Item	Estimated Cost	Description
Personal Computer or Laptop	Ksh.0.00	Processor (CPU), RAM (8GB minimum), Storage (SSD 256GB)
Operating System	Ksh.0.00	Windows
Internet Connection (Wi-Fi/Ethernet)	Ksh.0.00	School WI-FI
TOTAL	Ksh.0.00	

Table 1.5.2 Hardware Requirements

1.6 Project Schedule Breakdown

	PROJECT MILESTONES					
WEEKS	Project Planning & Analysis (System Documentation : Coverpage & Chapter One)	Project Design & Modeling (System Documentation Chapter Two)	Project Development & Testing (System Documentation Chapter Three)	Project Deployment (System Documentation Chapter Three)	Final Touches of System Documentation (Preliminary Pages, Chapter Four & References)	Project Presentation
5-9 Jan						
12-16 Jan						
19-23 Jan						
26-30 Jan						
2-6 Feb						
9-13 Feb						
16-20 Feb						
23-27 Feb						
2-6 Mar						
8-13 Mar						
16-20 Mar						
23-27 Mar						
2 Apr						

Table 1.6 Project Schedule Breakdown

CHAPTER TWO: DESIGN AND MODELING

2.1 Introduction to Modelling

In this chapter, the reader will see the designs and models that were created to help visualize the system before starting development. These models show how different parts of the system work together and how data flows between them. Modelling the system first helped to plan everything clearly, avoid mistakes, and make the actual development process faster and easier to follow. It also made it easier to understand how the system would work as a whole.

2.2 User Interface Models

This Model shows how the system will look and how users will interact with it. It includes diagrams of the main screens, such as the login page, registration page, dashboard, and voting pages. By creating the UI model, it was possible to plan the layout, navigation, and design elements in advance, making the system easier to use and more visually organized. This also helped to identify potential usability issues before actual development, ensuring a smoother experience for the users.

2.2.1 Home Page

This page shows a briefly description about the system. It has a login button for both the student and the admin and also the register button. It also has a contact information where one can reach out incase of emergency or any challenge faced.

Welcome to Student Leaders Voting System

Secure, transparent, and real-time student leaders online elections.
Students can vote for candidates, view results, and admins can manage elections easily.

[Student Login](#)[Admin Login](#)[Register](#)

Contact Us

Have questions? Reach out to us:

Email: support@univote.com

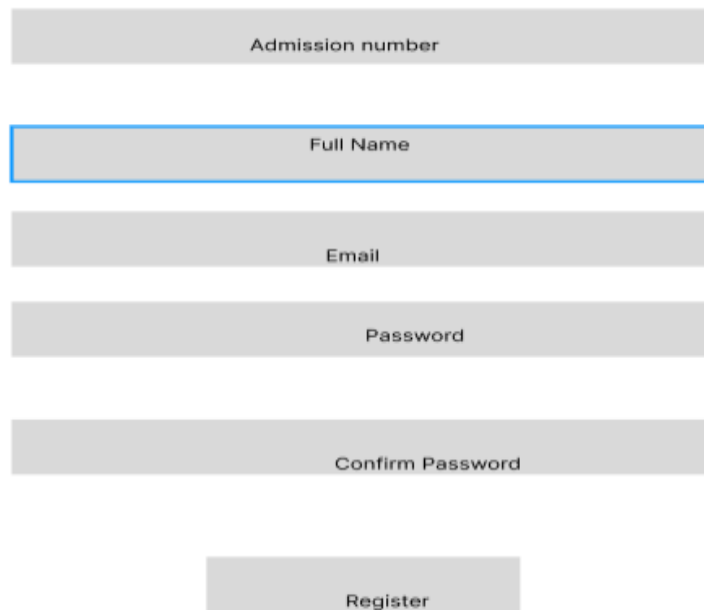
WhatsApp: [+254 114054646](https://wa.me/254114054646)

© 2020 Student Leaders Voting System. All rights reserved.

Fig 2.2.1 Home Page Design

2.2.2 Signup Page

This is the signup page where students get to register their details and choose the branch where they are based at. It has a register button and a link that redirects to the login if user is already registered.



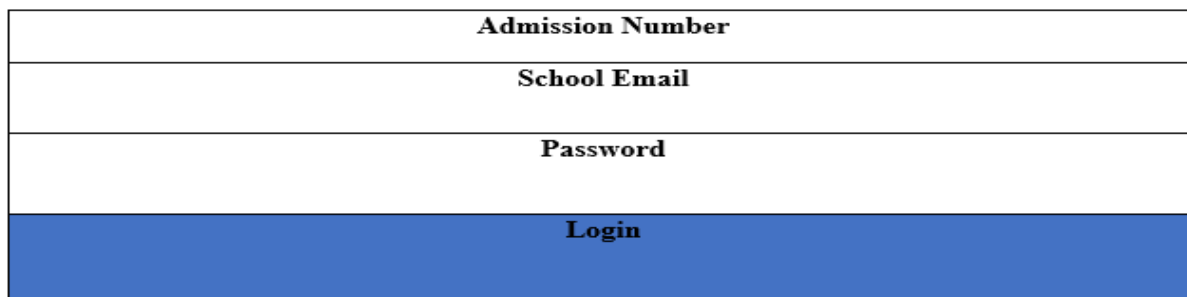
The image shows a vertical stack of five text input fields and a register button. The fields are labeled 'Admission number', 'Full Name', 'Email', 'Password', and 'Confirm Password'. The 'Full Name' field is highlighted with a blue border. Below the fields is a 'Register' button.

Admission number
Full Name
Email
Password
Confirm Password
Register

Fig 2.2.2 Signup Page Design

2.2.3 Login Page Design

2.2.3.1 Student Login



The image shows a login form with four rows. The first three rows are labeled 'Admission Number', 'School Email', and 'Password'. The fourth row is labeled 'Login' and has a blue background.

Admission Number
School Email
Password
Login

Fig 2.2.3.1 Student Login Page

Design Students get to login using their admission number and password.

2.2.3.2 Admin Login

Admin Username
Admin Password
Login

Fig 2.2.3.2 Admin Login Page Design

Admin login using the designated username and password.

2.2.4 Product Page Design

2.2.4.1 Student Dashboard

Student Dashboard
Cast Vote
View Results
Logout

Fig 2.2.4.1 Student Dashboard Page Design

The students get to cast their votes and are able to view results in real time.

2.2.4.2 Admin Dashboard

Admin Panel
Create Election
Manage Candidates
Manage Students
Monitor Voting
Publish Elections
View Results
Logout

Fig 2.2.4.2 Admin Dashboard Page Design

An administrator is able to create election, manage candidates, monitor voting view results and publish elections.

2.3 Logic Models

2.3.1 Use Case Diagrams

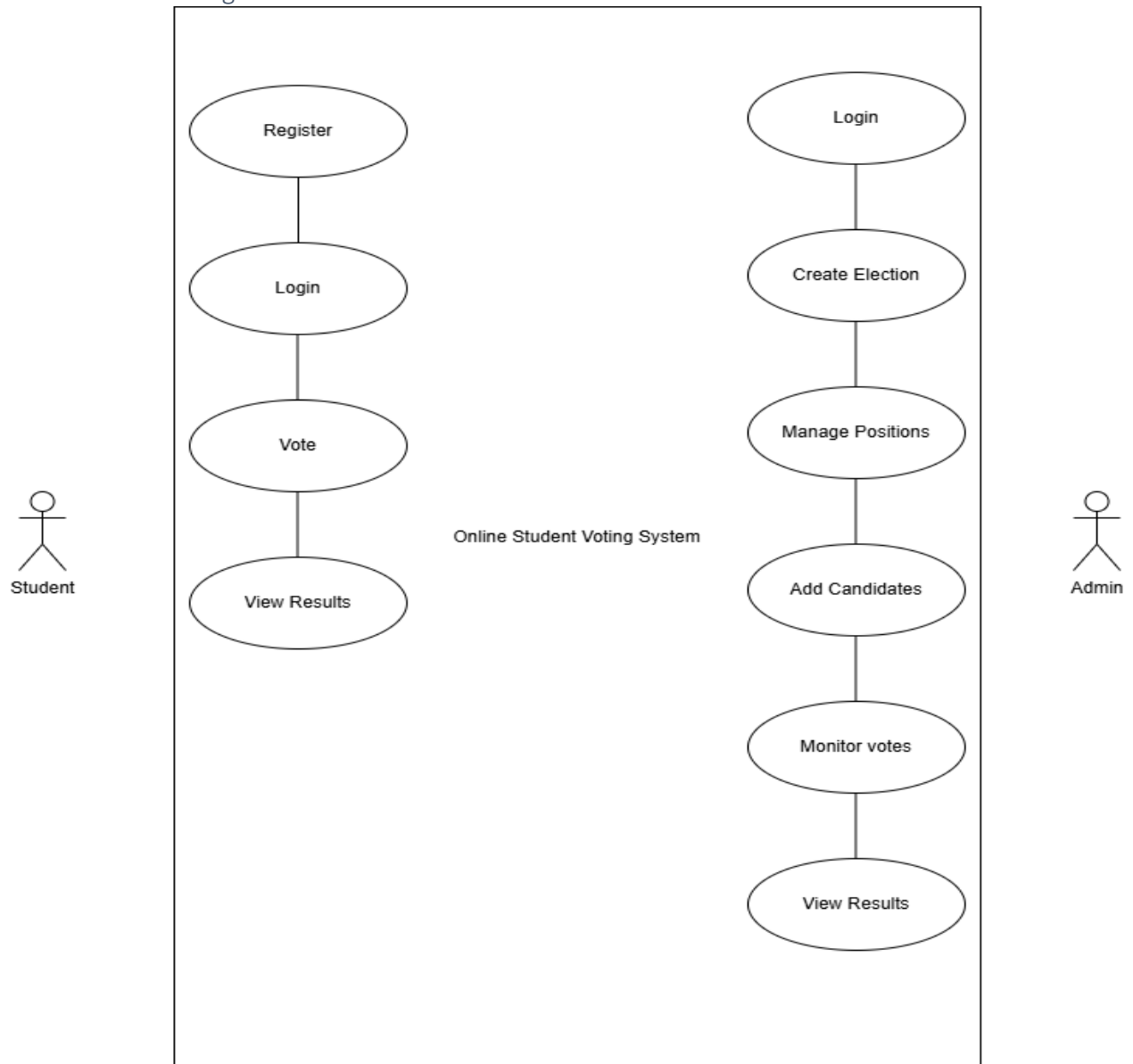


Fig 2.3.1 Use Case Diagram

The diagram shows interactions between the Admin and Voter. The Admin manages elections and candidates, while voters register, log in, and cast votes.

2.3.2 ER Diagrams

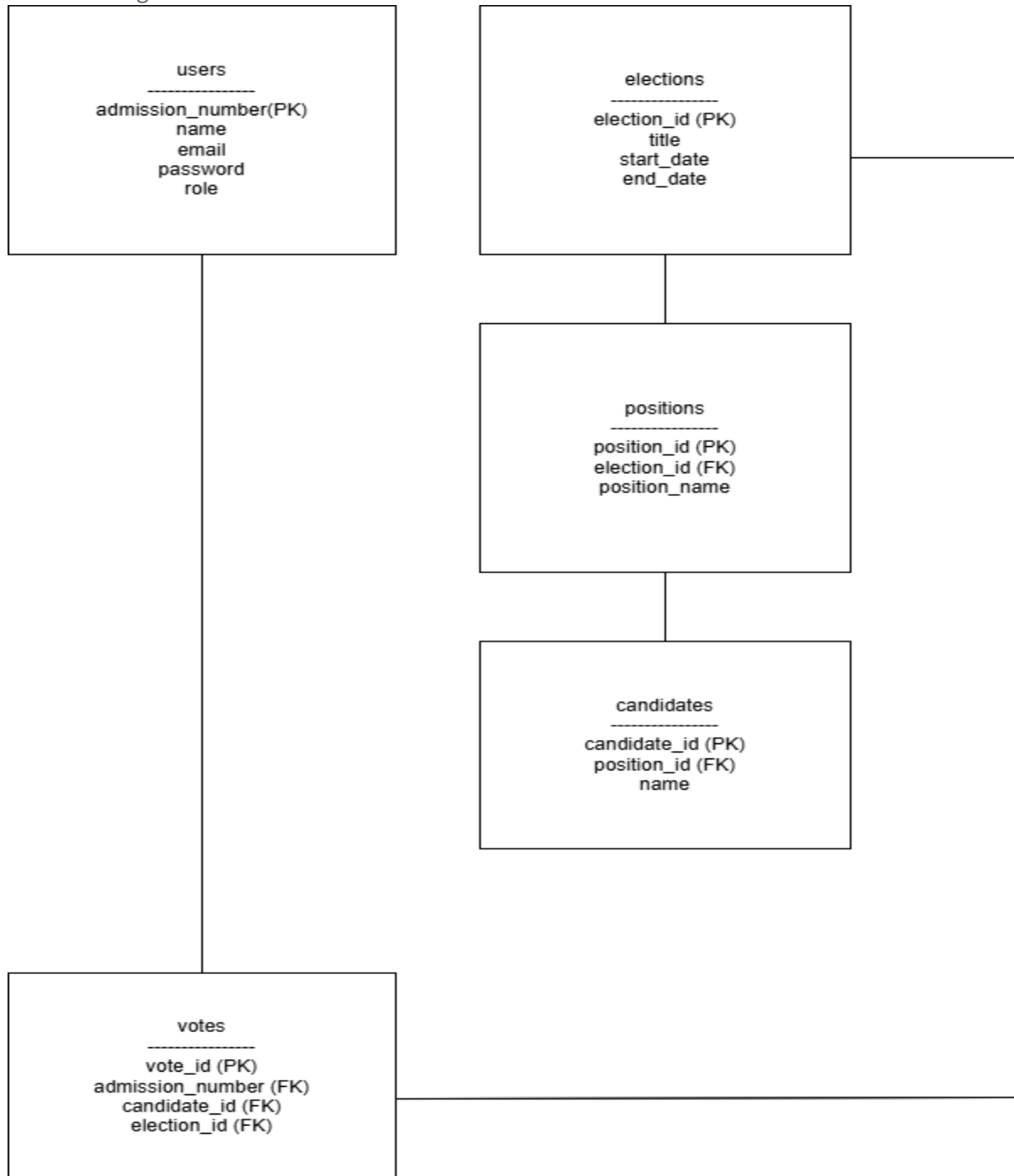


Figure 2.3.2: Entity Relationship Diagram for the Student Leaders Online Voting System
The ER diagram illustrates the database structure and relationships between users, elections, positions, candidates, and votes.

2.3.3 Flowcharts

2.3.3.1 Student flowchart

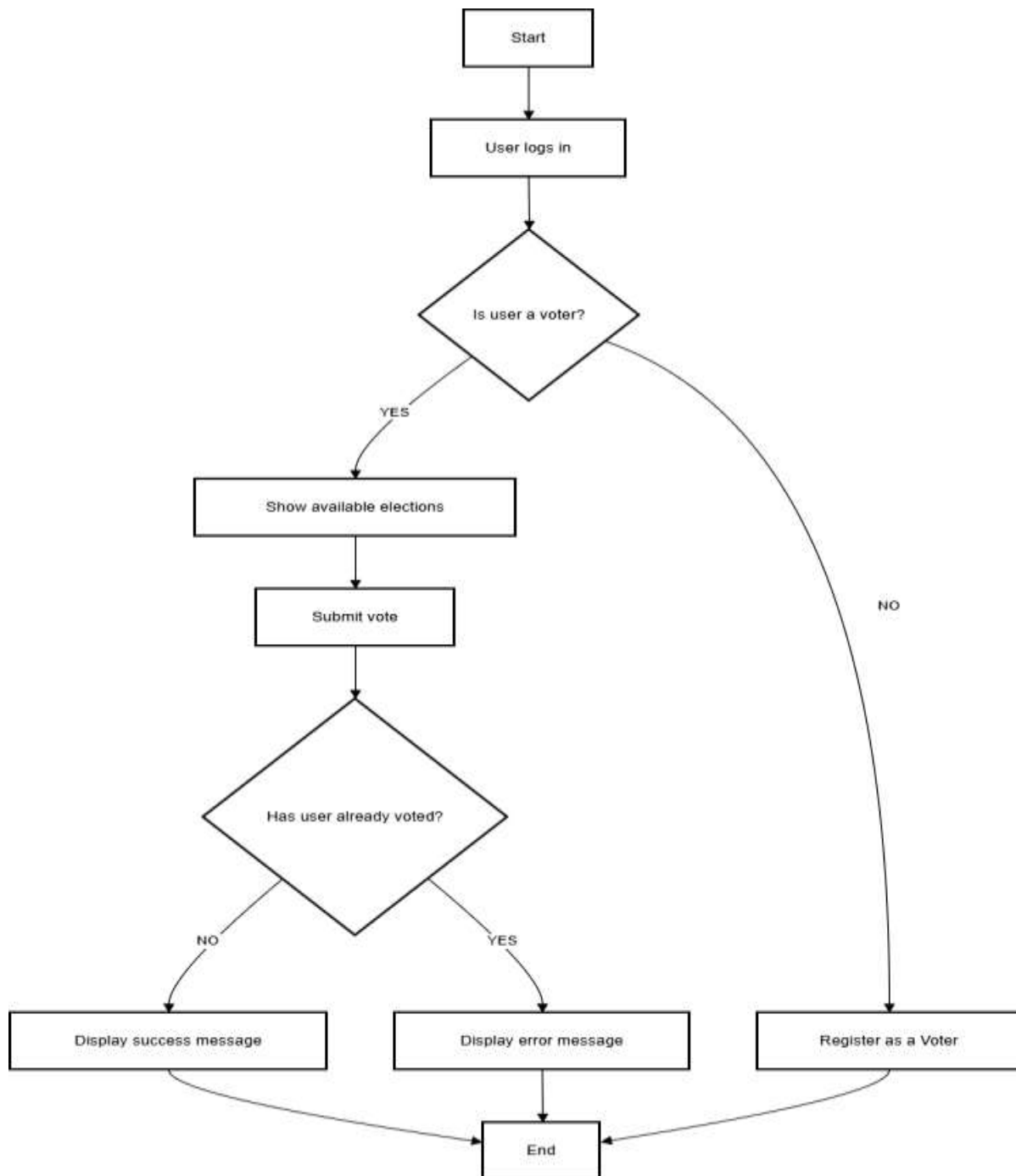


Figure 2.3.3.1: Voting Process Flowchart

The flowchart shows the steps followed when a voter logs in, selects a candidate, and casts a vote.

2.3.3.2 Admin Flowchart

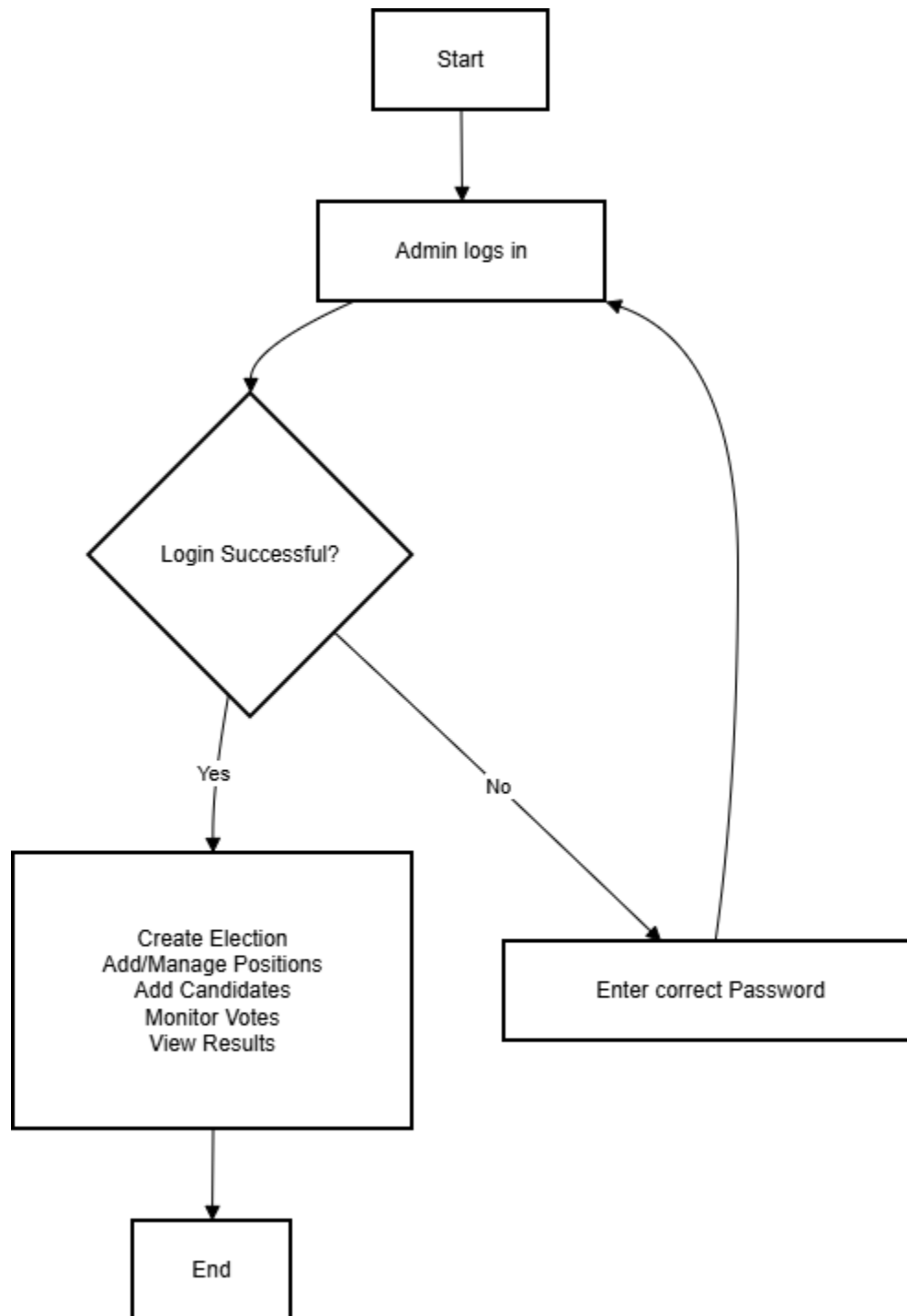


Figure 2.3.3.2: Admin Flowchart

The flowchart shows the process for the administrator, including login, creating elections, managing positions and candidates, monitoring votes, and viewing results

2.3.4 Architectural Diagrams

Web-Based System Architecture for the Student Leaders Voting System

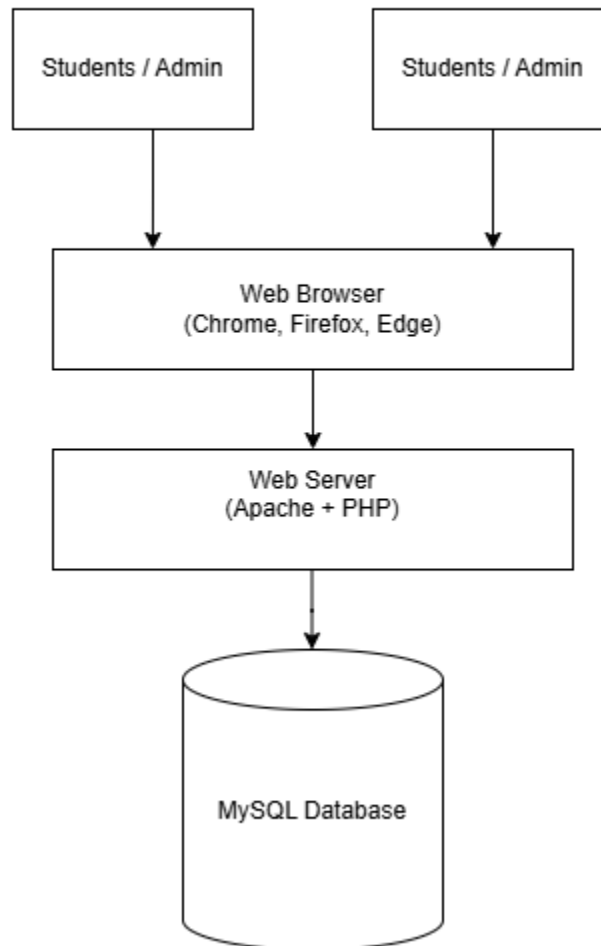


Figure 2.3.4: System Architecture Diagram

The diagram illustrates the web-based client-server architecture of the Student Leaders Online Voting System, showing interaction between users, the web server, and the database.

2.4 Class Diagrams

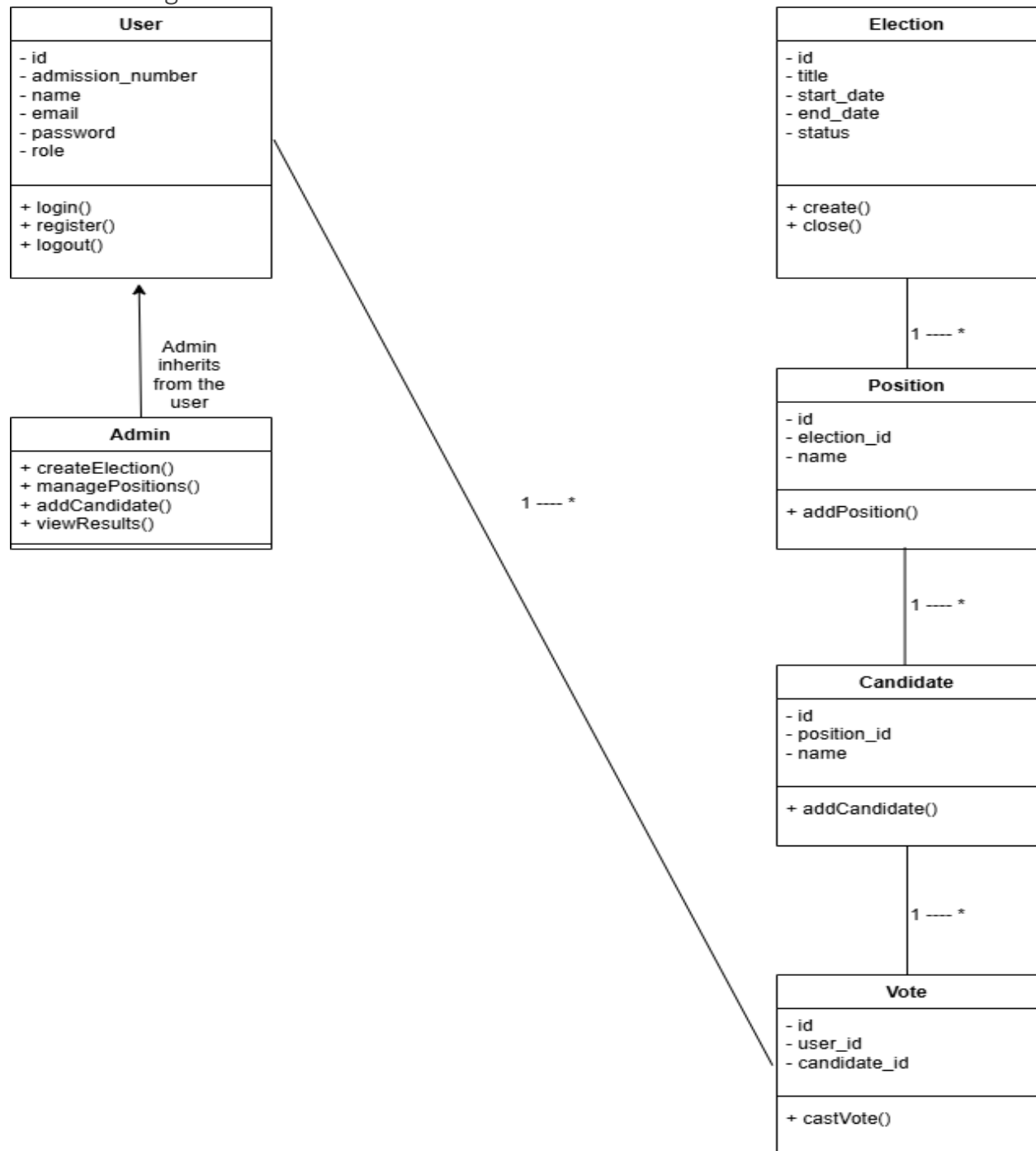


Figure 2.4: Class Diagram of the Online Student Voting System

This diagram illustrates the object-oriented structure of the system, showing classes, attributes, methods, and relationships between system components.

2.5 DFD Diagrams

2.5.1 Level 0

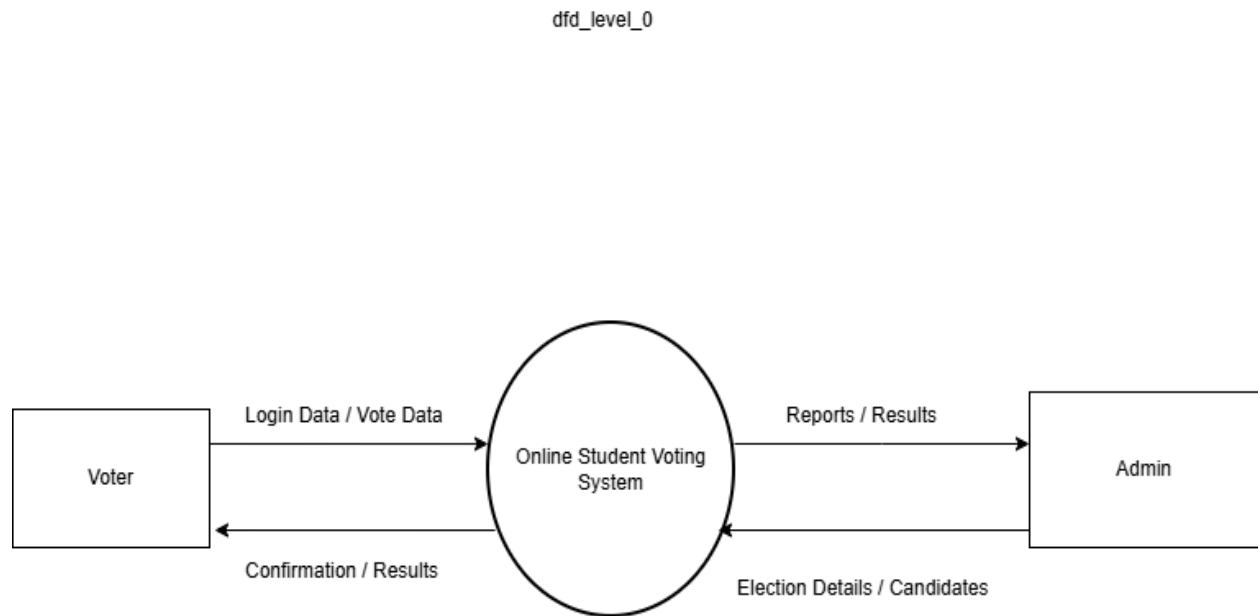


Figure 2.5.1: DFD Level 0 (Context Diagram)

This diagram represents the overall data flow between external entities (Voter and Admin) and the Online Student Voting System.

2.5.2 DFD Level 1

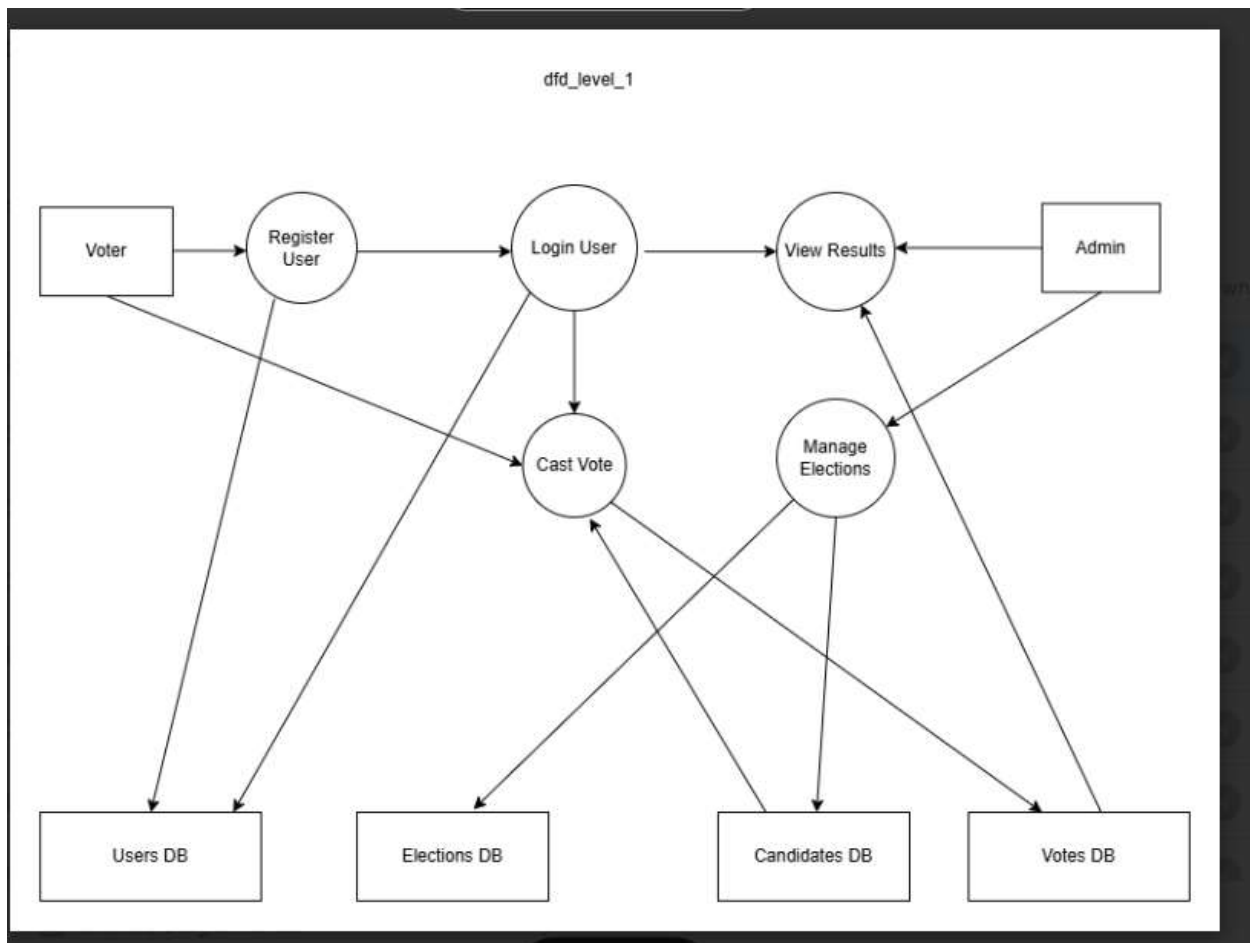


Figure 2.5.2: DFD Level 1

This diagram shows the internal processes of the Student Leaders Voting System, including user registration, login, election management, voting, and result generation, along with the associated data stores.

CHAPTER THREE: SYSTEM IMPLEMENTATION

(DEVELOPMENT, TESTING AND DEPLOYMENT)

3.1 Introduction

This chapter shows the development journey of the system, beginning with the setup of the development environment using Visual Studio Code due to its flexibility and support for multiple programming languages. It then describes the coding process; frontend and backend highlighting how the system's features were implemented and integrated. The chapter further explains how the database was designed and connected to support system functionality. Finally, it outlines the deployment process, showing how the system was made accessible to users. Overall, this chapter provides a clear, step-by-step overview of how the system was built from concept to a fully functional application.

3.2 User Interface Development

3.2.1 Login Page

3.2.1.1 Student login page

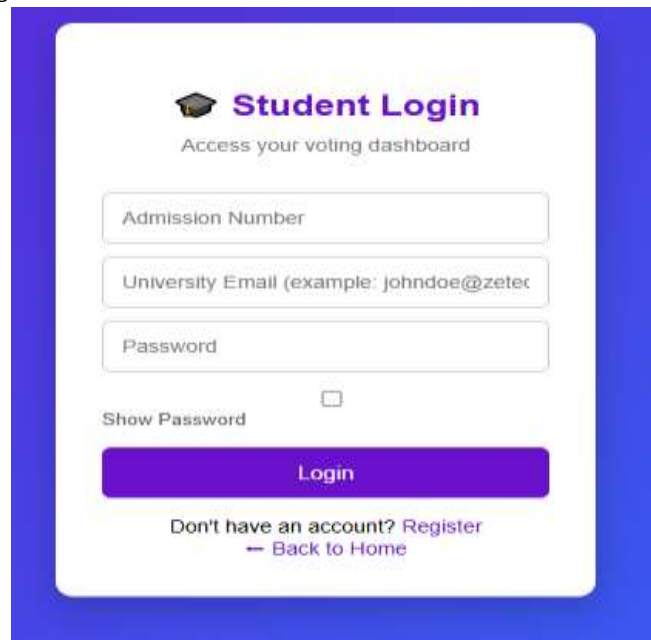
The image shows a 'Student Login' interface. At the top, there is a graduation cap icon followed by the text 'Student Login' in a bold, dark blue font. Below this is the subtitle 'Access your voting dashboard' in a smaller, grey font. The login form consists of three input fields: 'Admission Number', 'University Email (example: johndoe@zeter)', and 'Password'. The 'Password' field has a small square icon to its right, and below it is a 'Show Password' label. A large, solid blue button with the word 'Login' in white text is positioned below the input fields. At the bottom of the form, there is a link that says 'Don't have an account? Register' and a link with a left-pointing arrow that says 'Back to Home'.

Fig 3.2.1.1: Student Login Interface

Here, students get to login using their admission number and the password they used during the signing up. If the student doesn't have an account they are redirected to register.

3.2.1.2 Student Login Code

```
37 <!DOCTYPE html>
38 <html>
39 <head>
40   <title>Student Login</title>
41   <meta name="viewport" content="width=device-width, initial-scale=1.0">
42
43   <style>
44     * { box-sizing: border-box; font-family: Arial, sans-serif; }
45
46     body {
47       margin: 0;
48       height: 100vh;
49       background: linear-gradient(135deg, #6a11cb, #2575fc);
50       display: flex;
51       justify-content: center;
52       align-items: center;
53     }
54
55     .login-card {
56       background: white;
57       padding: 30px;
58       border-radius: 12px;
59       width: 350px;
60       box-shadow: 0 10px 25px rgba(0,0,0,0.2);
61       text-align: center;
62     }
63
64     h2 {
65       margin-bottom: 10px;
66       color: #6a11cb;
67     }
68
69     .subtitle {
70       font-size: 14px;
71       color: #777;
72       margin-bottom: 20px;
73     }
74   </style>
75 </head>
```

```

75     input {
76         width: 100%;
77         padding: 12px;
78         margin-top: 10px;
79         border-radius: 6px;
80         border: 1px solid #ccc;
81         font-size: 14px;
82     }
83
84     input:focus {
85         outline: none;
86         border-color: #6a11cb;
87     }
88
89     .btn {
90         width: 100%;
91         padding: 12px;
92         background: #6a11cb;
93         color: white;
94         border: none;
95         border-radius: 6px;
96         margin-top: 15px;
97         font-size: 15px;
98         cursor: pointer;
99         transition: 0.2s;
100    }
101
102    .btn:hover {
103        background: #4e0fa1;
104    }
105
106    .error {
107        background: #ffe5e5;
108        color: #d8000c;
109        padding: 8px;
110        margin-bottom: 10px;

```

```

111         border-radius: 6px;
112         font-size: 14px;
113     }
114
115     .links {
116         margin-top: 15px;
117         font-size: 14px;
118     }
119
120     .links a {
121         color: #6a11cb;
122         text-decoration: none;
123     }
124
125     .links a:hover {
126         text-decoration: underline;
127     }
128
129     .show-pass {
130         text-align: left;
131         font-size: 13px;
132         margin-top: 5px;
133         color: #555;
134     }
135 </style>
136 </head>
137 <body>
138
139 <div class="login-card">
140     <h2>🎓 Student Login</h2>
141     <div class="subtitle">Access your voting dashboard</div>
142
143     <?php if ($error): ?>
144         <div class="error"><?php echo $error; ?></div>
145     <?php endif; ?>

```

```

146
147     <form method="POST">
148         <input type="text" name="admission_number" placeholder="Admission Number" required>
149
150         <input type="email" name="email" placeholder="University Email (example: johndoe@zetech.ac.ke)" required>
151
152         <input type="password" id="password" name="password" placeholder="Password" required>
153
154         <div class="show-pass">
155             <input type="checkbox" onclick="togglePassword()"> Show Password
156         </div>
157
158         <button class="btn">Login</button>
159     </form>
160
161     <div class="links">
162         Don't have an account? <a href="register.php">Register</a><br>
163         <a href="index.php">⇐ Back to Home</a>
164     </div>
165 </div>
166
167 <script>
168 function togglePassword() {
169     var pass = document.getElementById("password");
170     pass.type = pass.type === "password" ? "text" : "password";
171 }
172 </script>
173
174 </body>
175 </html>
176

```

Fig 3.2.1.2: Student Login Code
shows the HTML and CSS code of the login page

3.2.1.3 Admin Login Page

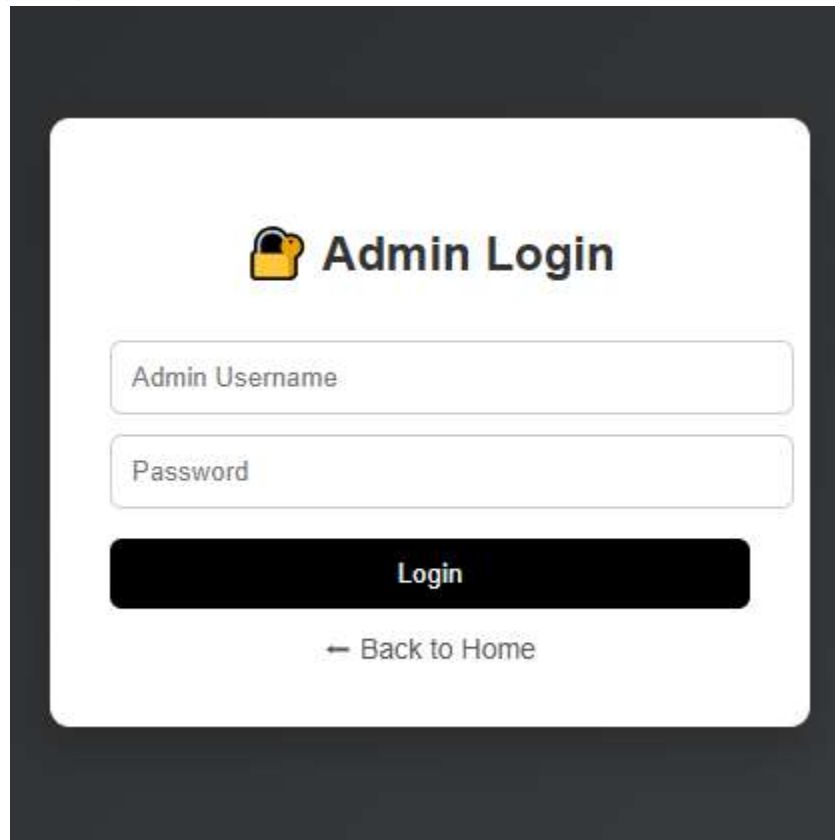
The image shows a web interface for an admin login page. It features a dark gray background with a white rounded rectangle in the center. Inside the rectangle, at the top, is a yellow padlock icon followed by the text "Admin Login" in a bold, black font. Below this, there are two white input fields with gray borders. The first field is labeled "Admin Username" and the second is labeled "Password". Below the input fields is a black button with the word "Login" in white text. At the bottom of the white rectangle, there is a link that says "← Back to Home" in a gray font.

Fig 3.2.1.3: Admin Login Interface

This page shows how admin get to log in to the system by using the already set username and password. Below it is the HTML and CSS part of admin page.

3.2.1.4 Admin login code

```
admin > login.php > ...
26
27 <!DOCTYPE html>
28 <html>
29 <head>
30   <title>Admin Login</title>
31   <style>
32     body {
33       margin: 0;
34       height: 100vh;
35       font-family: Arial;
36       background: linear-gradient(135deg, #232526, #414345);
37       display: flex;
38       justify-content: center;
39       align-items: center;
40     }
41
42     .card {
43       background: white;
44       padding: 30px;
45       border-radius: 10px;
46       width: 320px;
47       text-align: center;
48       box-shadow: 0 10px 25px rgba(0,0,0,0.2);
49     }
50
51     h2 { color: #333; }
52
53     input {
54       width: 100%;
55       padding: 10px;
56       margin-top: 10px;
57       border-radius: 6px;
58       border: 1px solid #ccc;
59     }
60
```

```

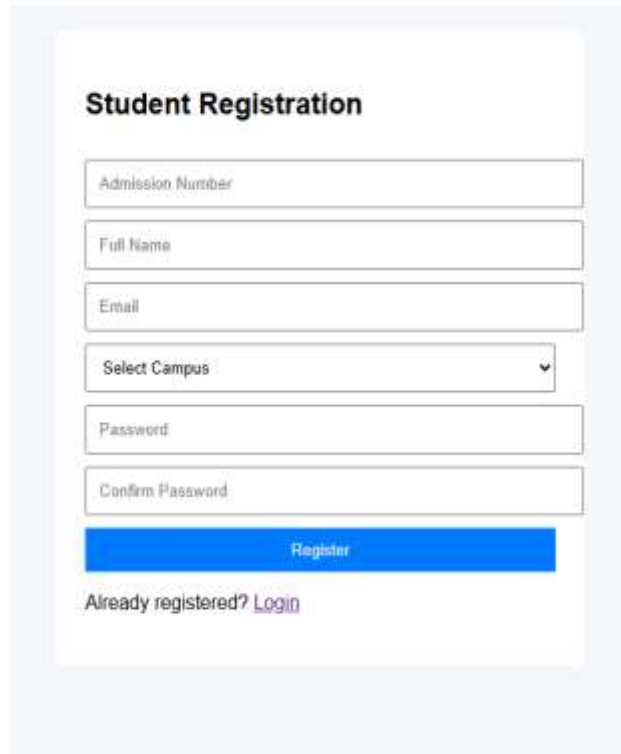
61     button {
62         width: 100%;
63         padding: 10px;
64         margin-top: 15px;
65         background: #000;
66         color: white;
67         border: none;
68         border-radius: 6px;
69         cursor: pointer;
70     }
71
72     .error {
73         color: red;
74         margin-bottom: 10px;
75     }
76
77     a {
78         display: block;
79         margin-top: 10px;
80         font-size: 14px;
81         text-decoration: none;
82         color: #555;
83     }
84 </style>
85 </head>
86 <body>
87
88 <div class="card">
89     <h2>🔒 Admin Login</h2>
90
91     <?php if ($error): ?>
92         <div class="error"><?php echo $error; ?></div>
93     <?php endif; ?>
94
95     <form method="POST">
96         <input type="text" name="username" placeholder="Admin Username" required>
97         <input type="password" name="password" placeholder="Password" required>
98         <button>Login</button>
99     </form>
100
101     <!-- File is inside /admin/, index.php is one level up -->
102     <a href="../index.php">← Back to Home</a>
103 </div>
104
105 </body>
106 </html>

```

Fig 3.2.1.4: Admin login code

Shows the HTML and CSS part of the login page.

3.2.2 Signup page

A screenshot of a web form titled "Student Registration". The form is contained within a white box with a light blue border. It features several input fields: "Admission Number", "Full Name", "Email", "Select Campus" (a dropdown menu), "Password", and "Confirm Password". Below these fields is a prominent blue button labeled "Register". At the bottom of the form, there is a link that says "Already registered? Login".

Student Registration

Admission Number

Full Name

Email

Select Campus

Password

Confirm Password

Register

Already registered? [Login](#)

Fig 3.2.2: Signup Page

This page allows only the students to register using their admission number, Full name, valid email, select which branch campus they're at and password. Below it is the code snippet of the page.

3.2.3 Signup page code

```
55 <!DOCTYPE html>
56 <html>
57 <head>
58   <title>Student Registration</title>
59   <style>
60     body {
61       font-family: Arial;
62       background: #f4f6f9;
63     }
64     .box {
65       width: 400px;
66       margin: 50px auto;
67       padding: 25px;
68       background: white;
69       border-radius: 8px;
70     }
71     input, select, button {
72       width: 100%;
73       padding: 10px;
74       margin-top: 10px;
75     }
76     button {
77       background: #007bff;
78       color: white;
79       border: none;
80     }
81     .msg {
82       color: red;
83       margin-top: 10px;
84     }
85   </style>
86 </head>
87 <body>
88
89 <div class="box">
90   <h2>Student Registration</h2>
91
92   <?php if ($message) echo "<div class='msg'>$message</div>"; ?>
```

```

89 <div class="box">
90     <h2>Student Registration</h2>
91
92     <?php if ($message) echo "<div class='msg'>$message</div>"; ?>
93
94     <form method="POST">
95         <input type="text" name="admission" placeholder="Admission Number" required>
96
97         <input type="text" name="name" placeholder="Full Name" required>
98
99         <input type="email" name="email" pattern=".+@zetech\.ac\.ke"
100 placeholder="University Email (example: johndoe@zetech.ac.ke)" required>
101
102         <select name="campus" required>
103             <option value="">Select Campus</option>
104             <option>Mangu</option>
105             <option>TRC</option>
106             <option>Town</option>
107         </select>
108
109         <input type="password" name="password" placeholder="Password" required>
110
111         <input type="password" name="confirm" placeholder="Confirm Password" required>
112
113         <button type="submit">Register</button>
114     </form>
115
116     <p>Already registered? <a href="login.php">Login</a></p>
117 </div>
118
119 </body>
120 </html>
121

```

Fig 3.2.3: Signup page code
shows the HTML and CSS part of the register page.

3.2.4 Dashboards

3.2.4.1. Admin Interface

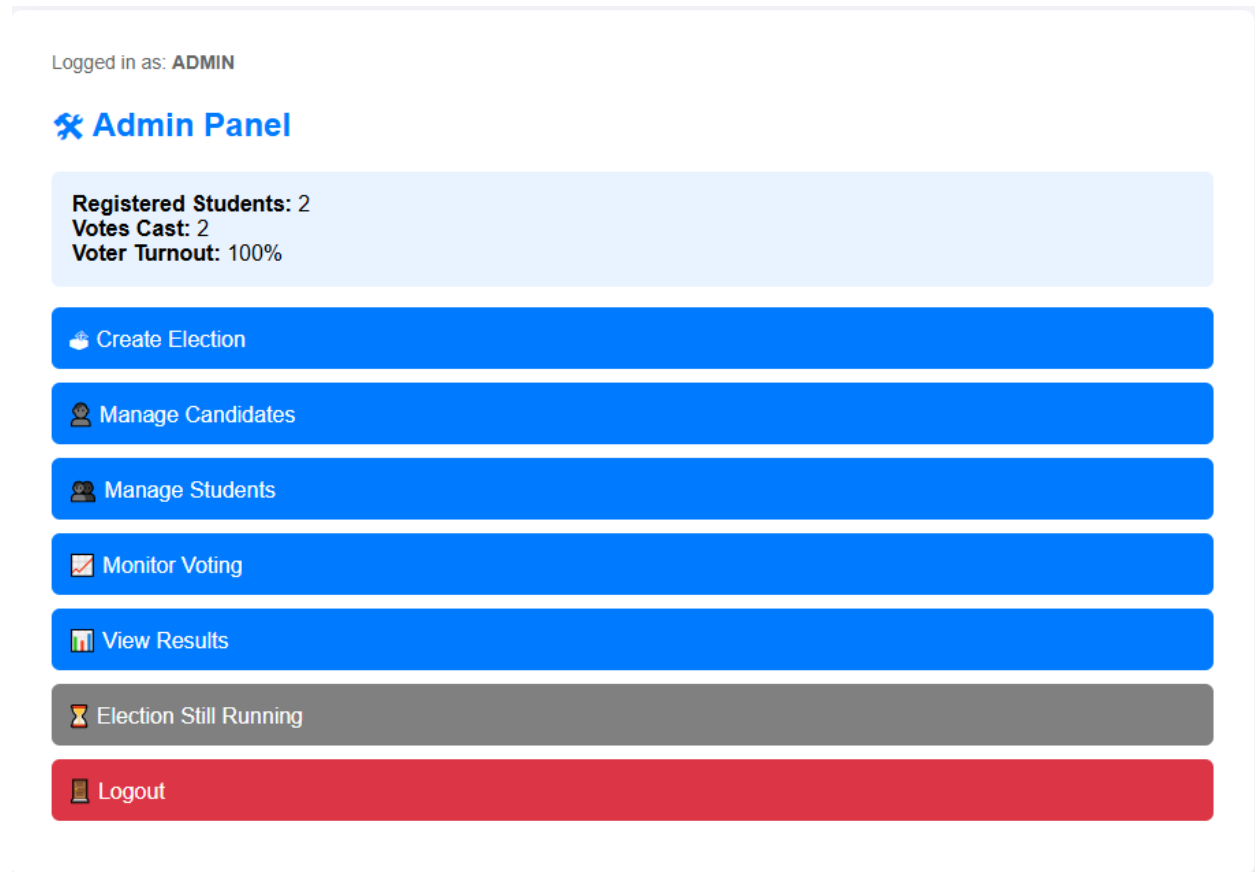
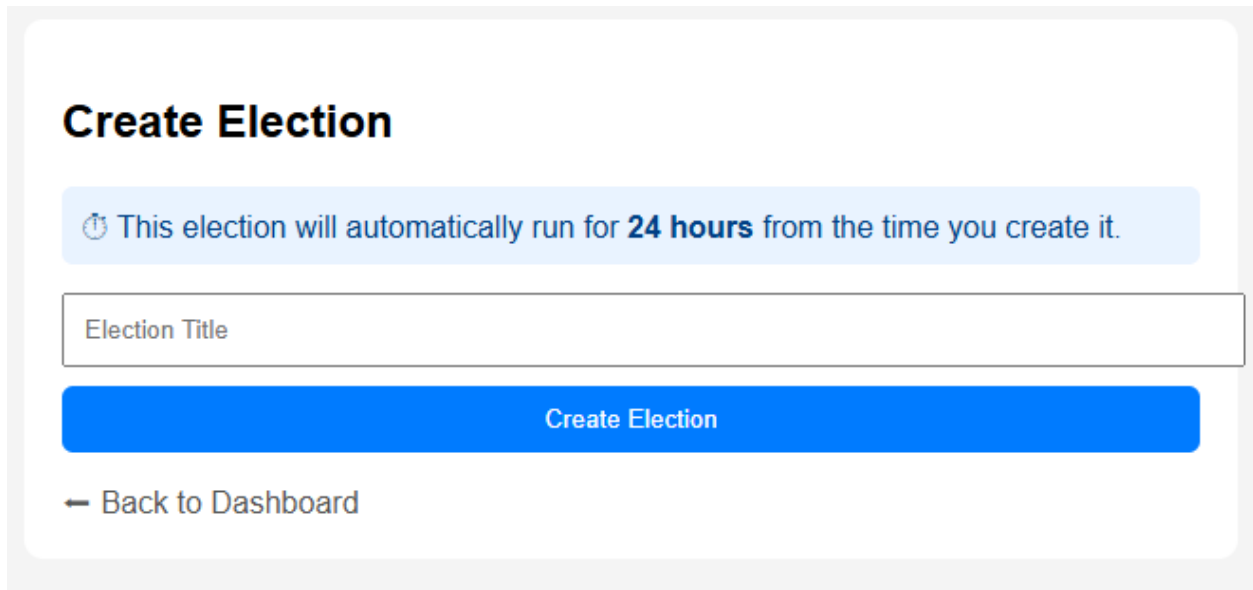


Fig 3.2.4.1: Admin Interface

After successful login, the admin gets to create election, manage candidates, monitor voting and view results of the elections. An admin also gets to publish the results after the election.

3.2.4.1. Create Election Interface



The image shows a web interface for creating an election. It features a title 'Create Election' in bold black text. Below the title is a light blue informational box with a clock icon and the text 'This election will automatically run for 24 hours from the time you create it.' Underneath this is a white input field with the placeholder text 'Election Title'. Below the input field is a prominent blue button with the text 'Create Election' in white. At the bottom left, there is a link with a left-pointing arrow and the text 'Back to Dashboard'.

Fig 3.2.4.1.1: Create Election Interface

This page allows the admin to create election which will automatically end after 24hrs from the created time.

3.2.4.2 Dashboard Interface Code

```
dashboard.php > html
84 <!DOCTYPE html>
85 <html>
86 <head>
87   <title>Dashboard</title>
88   <meta name="viewport" content="width=device-width, initial-scale=1.0">
89
90   <style>
91     body { font-family: Arial; background: #f4f6f9; }
92
93     .container {
94       max-width: 850px;
95       width: 90%;
96       margin: 50px auto;
97       background: white;
98       padding: 30px;
99       border-radius: 8px;
100      box-shadow: 0 8px 20px rgba(0,0,0,0.1);
101    }
102
103    h2 { color: #007bff; }
104
105    .btn {
106      display: block;
107      margin: 10px 0;
108      padding: 12px;
109      background: #007bff;
110      color: white;
111      text-decoration: none;
112      border-radius: 6px;
113    }
114
115    .btn:hover { background: #0056b3; }
116
117    .logout { background: #dc3545; }
118  </style>
```

```

118
119     .success {
120         background:#d4edda;
121         color:#155724;
122         padding:12px;
123         border-radius:6px;
124         margin-bottom:15px;
125     }
126
127     .warning {
128         color: red;
129         font-weight: bold;
130     }
131
132     .disabled {
133         background: grey !important;
134         pointer-events: none;
135     }
136
137     .role {
138         font-size: 14px;
139         color: #666;
140         margin-bottom: 15px;
141     }
142 </style>
143 </head>
144
145 <body>
146
147 <div class="container">
148
149     <!-- ROLE -->
150     <div class="role">
151         | Logged in as: <strong><?php echo strtoupper($role); ?></strong>
152     </div>

```

```

154 <!-- SUCCESS -->
155 <?php if (isset($_GET['published'])): ?>
156 |     <div class="success">🔊 Results have been successfully published!</div>
157 <?php endif; ?>
158
159 <!-- ERROR POPUPS -->
160 <?php if (isset($_GET['error'])): ?>
161 <script>
162 |     <?php if ($_GET['error'] == 'not_ended'): ?>
163 |         alert("⌚ Election has not ended yet!");
164 |     <?php elseif ($_GET['error'] == 'already_published'): ?>
165 |         alert("⚠️ Results already published!");
166 |     <?php elseif ($_GET['error'] == 'failed'): ?>
167 |         alert("❌ Failed to publish results.");
168 |     <?php elseif ($_GET['error'] == 'no_id'): ?>
169 |         alert("❌ No election selected.");
170 |     <?php elseif ($_GET['error'] == 'not_found'): ?>
171 |         alert("❌ Election not found.");
172 |     <?php endif; ?>
173 </script>
174 <?php endif; ?>
175
176
177 <!-- ===== STUDENT ===== -->
178 <?php if ($role === "student"): ?>
179 |     <h2>👤 Student Dashboard</h2>
180
181 |     <?php if ($alreadyVoted): ?>
182 |         <a class="btn disabled">✓ Vote Submitted</a>
183 |     <?php else: ?>
184 |         <a href="vote.php" class="btn">🗳️ Cast Vote</a>
185 |     <?php endif; ?>
186
187 |     <a href="results.php" class="btn">📊 View Results</a>

```

```

87     <a href="results.php" class="btn"><img alt="Bar chart icon" data-bbox="488 95 500 105"/> View Results</a>
88
89 <?php endif; ?>
90
91
92 <!-- ===== ADMIN ===== -->
93 <?php if ($role === "admin"): ?>
94     <h2><img alt="Key icon" data-bbox="221 198 233 208"/> Admin Panel</h2>
95
96     <div style="background:#e9f3ff;padding:15px;border-radius:6px;margin-bottom:15px;">
97         <strong>Registered Students:</strong> <?php echo $totalStudents; ?><br>
98         <strong>Votes Cast:</strong> <?php echo $totalVotes; ?><br>
99         <strong>Voter Turnout:</strong> <?php echo $turnout; ?>%
100     </div>
101
102     <a href="admin/create_election.php" class="btn"><img alt="Flag icon" data-bbox="408 315 420 325"/> Create Election</a>
103     <a href="admin/manage_candidates.php" class="btn"><img alt="Person icon" data-bbox="408 330 420 340"/> Manage Candidates</a>
104     <a href="admin/manage_students.php" class="btn"><img alt="Group of people icon" data-bbox="408 345 420 355"/> Manage Students</a>
105     <a href="admin/monitor_votes.php" class="btn"><img alt="Checkmark icon" data-bbox="478 360 490 370"/> Monitor Voting</a>
106     <a href="results.php" class="btn"><img alt="Bar chart icon" data-bbox="488 375 500 385"/> View Results</a>
107
108 <!-- PUBLISH BUTTON LOGIC -->
109 <?php if ($election_id): ?>
110
111     <?php if ($election['results_published'] == 1): ?>
112         <a class="btn disabled"><img alt="Checkmark icon" data-bbox="281 462 293 472"/> Results Published</a>
113
114     <?php elseif ($election['end_date'] > $current): ?>
115         <a class="btn disabled"><img alt="Hourglass icon" data-bbox="281 505 293 515"/> Election Still Running</a>
116
117     <?php else: ?>
118         <a href="admin/publish_results.php?id=<?php echo $election_id; ?>"
119             class="btn"
120             onclick="return confirm('Publish official election results?')">
121             <img alt="Megaphone icon" data-bbox="308 595 320 605"/> Publish Final Results
122         </a>

```

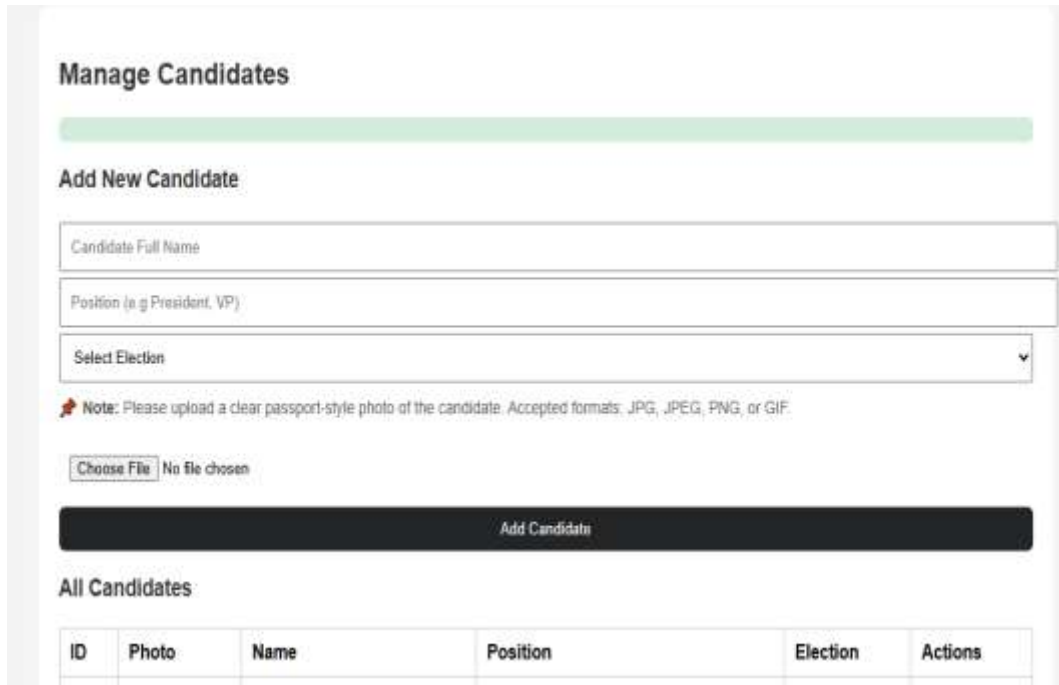
```

223         <?php endif; ?>
224
225     <?php else: ?>
226         <a class="btn disabled">⚠ No Election Found</a>
227     <?php endif; ?>
228
229 <?php endif; ?>
230
231 <!-- LOGOUT -->
232 <a href="logout.php" class="btn logout">🚪 Logout</a>
233
234 </div>
235
236 </body>
237 </html>

```

Fig 3.2.4.2: Dashboard Interface Code
Shows the HTML and CSS part of the dashboard interface.

3.2.4.1.2 Manage Candidates Interface



The interface is titled "Manage Candidates" and features a green header bar. Below the header, there is a section for "Add New Candidate" with three input fields: "Candidate Full Name", "Position (e.g. President, VP)", and "Select Election" (a dropdown menu). A note with a red icon states: "Note: Please upload a clear passport-style photo of the candidate. Accepted formats: JPG, JPEG, PNG, or GIF". Below the note is a file upload section with a "Choose File" button and the text "No file chosen". A large black button labeled "Add Candidate" is positioned below the form. At the bottom, there is a section titled "All Candidates" containing a table with the following columns: ID, Photo, Name, Position, Election, and Actions.

ID	Photo	Name	Position	Election	Actions
----	-------	------	----------	----------	---------

Fig 3.2.4.1.2: Manage Candidates Interface

Here the admin is able to manage the candidates by adding and deleting candidates.

3.2.4.1.3 Manage Candidate Code

```
98 <!DOCTYPE html>
99 <html>
100 <head>
101     <title>Manage Candidates</title>
102     <style>
103         body { font-family: Arial; background: #f4f4f4; padding: 20px; }
104         .container { max-width: 950px; margin: auto; background: white; padding: 20px; border-radius: 10px; }
105         h2, h3 { color: #333; }
106         form input, form select, form button { padding: 10px; margin-top: 5px; width: 100%; }
107         form button { cursor: pointer; background: #232526; color: white; border: none; border-radius: 6px; margin-top: 10px; }
108         table { width: 100%; border-collapse: collapse; margin-top: 20px; }
109         table, th, td { border: 1px solid #ccc; }
110         th, td { padding: 10px; text-align: left; }
111         .message { background: #d4edda; color: #155724; padding: 10px; margin: 10px 0; border-radius: 6px; }
112         a.delete { color: red; text-decoration: none; }
113         a.delete:hover { text-decoration: underline; }
114         a.back { border-radius: 6px; }
115         a.back { margin-top: 15px; display: inline-block; text-decoration: none; color: #555; }
116     </style>
117 </head>
118 <body>
119
120 <div class="container">
121     <h2>Manage Candidates</h2>
122
123     <?php if (isset($message)) echo "<div class='message'>$message</div>"; ?>
124
125     <h3>Add New Candidate</h3>
126     <form method="POST" enctype="multipart/form-data">
127
128         <input type="text" name="name" placeholder="Candidate Full Name" required>
129         <input type="text" name="position" placeholder="Position (e.g President, VP)" required>
130
131         <!-- Election dropdown -->
132         <select name="election_id" required>
133             <option value="">Select Election</option>
134             <?php while($row = $elections->fetch_assoc()); ?>
135                 <option value="<?php echo $row['id']; ?>"><?php echo htmlspecialchars($row['title']); ?></option>
136             <?php endwhile; ?>
137     </form>
138 </div>
```

```

137     </select>
138
139     <!-- Photo upload note -->
140     <p style="color:#555; font-size:14px; margin-top:10px;">
141     ✖ <strong>Note:</strong> Please upload a clear passport-style photo of the candidate.
142     Accepted formats: JPG, JPEG, PNG, or GIF.
143     </p>
144
145     <input type="file" name="photo" accept="image/*" required>
146
147     <button name="add_candidate">Add Candidate</button>
148     </form>
149
150     <h3>All Candidates</h3>
151     <table>
152     <tr>
153     <th>ID</th>
154     <th>Photo</th>
155     <th>Name</th>
156     <th>Position</th>
157     <th>Election</th>
158     <th>Actions</th>
159     </tr>
160     <?php while($row = $candidates->fetch_assoc()): ?>
161     <tr>
162     <td><?php echo $row['id']; ?></td>
163     <td>
164     <?php if($row['photo'] && file_exists('../uploads/'.$row['photo'])): ?>
165     
166     <?php else: ?>
167     No photo
168     <?php endif; ?>
169     </td>
170     <td><?php echo htmlspecialchars($row['name']); ?></td>
171     <td><?php echo htmlspecialchars($row['position']); ?></td>
172     <td><?php echo htmlspecialchars($row['election_title']); ?></td>
173     <td>
174     <a class="delete" href="?delete=<?php echo $row['id']; ?>" onclick="return confirm('Delete this candidate?')">Delete
175     </td>
176     </tr>
177     <?php endwhile; ?>
178     </table>
179
180     <a class="back" href="../../dashboard.php">← Back to Dashboard</a>
181 </div>
182
183 </body>
184 </html>
185

```

Fig 3.2.4.1.3: Manage Candidates Interface Code
Shows the HTML and CSS part of the manage candidate page.

3.2.4.1.4 Manage Students Interface

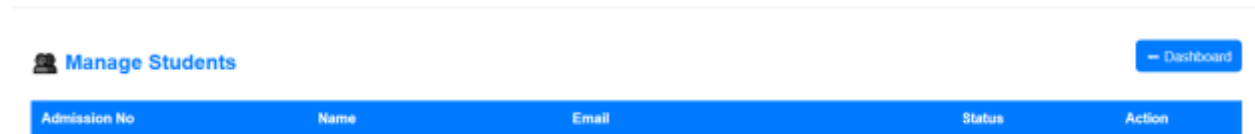


Fig 3.2.4.1.4: Manage Students Interface

This allows the admin to verify the student credentials and then approve or deny access to the dashboard. Below it is the HTML & CSS codes for the page.

3.2.4.1.5 Manage Students Interface Code

```
13
14 <!DOCTYPE html>
15 <html>
16 <head>
17   <title>Manage Students</title>
18
19   <style>
20     body {
21       font-family: Arial;
22       background: #f4f6f9;
23     }
24
25     .container {
26       width: 90%;
27       margin: 40px auto;
28       background: white;
29       padding: 25px;
30       border-radius: 8px;
31       box-shadow: 0 8px 20px rgba(0,0,0,0.1);
32     }
33
34     h2 {
35       color: #007bff;
36       margin-bottom: 10px;
37     }
38
39     .top-bar {
40       display: flex;
41       justify-content: space-between;
42       align-items: center;
43       margin-bottom: 20px;
44     }
45
```

```

46     .btn-dashboard {
47         padding: 8px 12px;
48         background: #007bff;
49         color: white;
50         text-decoration: none;
51         border-radius: 6px;
52     }
53
54     .btn-dashboard:hover {
55         background: #0056b3;
56     }
57
58     table {
59         width: 100%;
60         border-collapse: collapse;
61     }
62
63     th, td {
64         padding: 12px;
65         text-align: left;
66         border-bottom: 1px solid #ddd;
67     }
68
69     th {
70         background: #007bff;
71         color: white;
72     }
73
74     tr:hover {
75         background: #f1f1f1;
76     }
77
78     .btn {
79         padding: 6px 10px;
80         border-radius: 5px;
81         text-decoration: none;

```

```

82         color: white;
83         font-size: 13px;
84     }
85
86     .approve {
87         background:  #28a745;
88     }
89
90     .reject {
91         background:  #dc3545;
92     }
93
94     .status {
95         padding: 5px 10px;
96         border-radius: 5px;
97         color: white;
98         font-size: 12px;
99     }
100
101     .pending { background: orange; }
102     .approved { background: green; }
103     .rejected { background: red; }
104
105 </style>
106 </head>
107
108 <body>
109
110 <div class="container">
111
112     <!-- Top Section -->
113     <div class="top-bar">
114         <h2>👤 Manage Students</h2>
115         <a href="../../dashboard.php" class="btn-dashboard">⬅ Dashboard</a>
116     </div>

```

```

118 <table>
119 <tr>
120 <th>Admission No</th>
121 <th>Name</th>
122 <th>Email</th>
123 <th>Status</th>
124 <th>Action</th>
125 </tr>
126
127 <?php while($row = $result->fetch_assoc()): ?>
128 <tr>
129 <td><?php echo $row['admission_number']; ?></td>
130 <td><?php echo htmlspecialchars($row['full_name']); ?></td>
131 <td><?php echo htmlspecialchars($row['email']); ?></td>
132
133 <td>
134 <span class="status <?php echo $row['status']; ?>">
135 <?php echo ucfirst($row['status']); ?>
136 </span>
137 </td>
138
139 <td>
140 <?php if ($row['status'] === 'pending'): ?>
141 <a class="btn approve"
142 href="approve.php?id=<?php echo urlencode($row['admission_number']); ?>">
143 <img alt="checkmark icon" data-bbox="338 433 353 443"/> Approve
144 </a>
145
146 <a class="btn reject"
147 href="reject.php?id=<?php echo urlencode($row['admission_number']); ?>">
148 <img alt="cross icon" data-bbox="338 500 353 510"/> Reject
149 </a>
150 <?php else: ?>
151 -
152 <?php endif; ?>
153 </td>
154 </tr>
155 <?php endwhile; ?>
156
157 </table>
158
159 </div>
160
161 </body>
162 </html>

```

Fig 3.2.4.1.5: Manage Students Code
shows the HTML and CSS part of the manage student page.

3.2.4.1.6 Monitor Voting Interface

Monitor Voting				
Total Votes Cast: 4				
ID	Photo	Name	Position	Votes
19		VIOLA DAVIS	PRESIDENT	1
17		DILRABA	PRESIDENT	1

Fig 3.2.4.1.6: Monitor Voting Interface

Here the admin is able to monitor the progress of the elections.

3.2.4.1.7 Monitor Voting Interface Code

```
26 <docType html>
27 <html>
28 <head>
29 <title>Monitor Voting</title>
30 <style>
31 body { font-family: Arial; background: #f0f0f0; padding: 20px; }
32 .container { max-width: 950px; margin: auto; background: white; padding: 20px; border-radius: 10px; }
33 h2 { color: #333; }
34 table { width: 100%; border-collapse: collapse; margin-top: 20px; }
35 table, th, td { border: 1px solid #ccc; }
36 th, td { padding: 10px; text-align: left; }
37 img { border-radius: 5px; }
38 .total { margin-top: 10px; font-weight: bold; }
39 a.back { margin-top: 15px; display: inline-block; text-decoration: none; color: #555; }
40 </style>
41 </head>
42 <body>
43
44 <div class="container">
45 <h2>Monitor Voting</h2>
46
47 <div class="total">Total Votes Cast: <php echo $totalVotes; ?></div>
48
49 <table>
50 <tr>
51 <th>ID</th>
52 <th>Photo</th>
53 <th>Name</th>
54 <th>Position</th>
55 <th>Votes</th>
56 </tr>
57 <php while($row = $result->fetch_assoc()); ?>
58 <tr>
59 <td><php echo $row['id']; ?></td>
60 <td>
61 <php if($row['photo'] && file_exists('../uploads/'.$row['photo'])): ?>
62 <img src='../uploads/<php echo $row['photo']; ?>' width="60" alt="Photo">
63 <php else: ?>
```

```

63         <?php else: ?>
64             No photo
65         <?php endif; ?>
66     </td>
67     <td><?php echo htmlspecialchars($row['name']); ?></td>
68     <td><?php echo htmlspecialchars($row['position']); ?></td>
69     <td><?php echo $row['votes']; ?></td>
70 </tr>
71 <?php endwhile; ?>
72 </table>
73
74     <a class="back" href="../dashboard.php">← Back to Dashboard</a>
75 </div>
76
77 </body>
78 </html>
79 |

```

Fig 3.2.4.1.7: Monitor Voting Interface Code
shows the HTML and CSS part of the monitor voting page.

3.2.4.1.8 View Results Interface



Fig 3.2.4.1.8: View Results Interface

This page allows the admin to view results in real time.

3.2.5 Student Dashboard

3.2.5.1 Student Interface

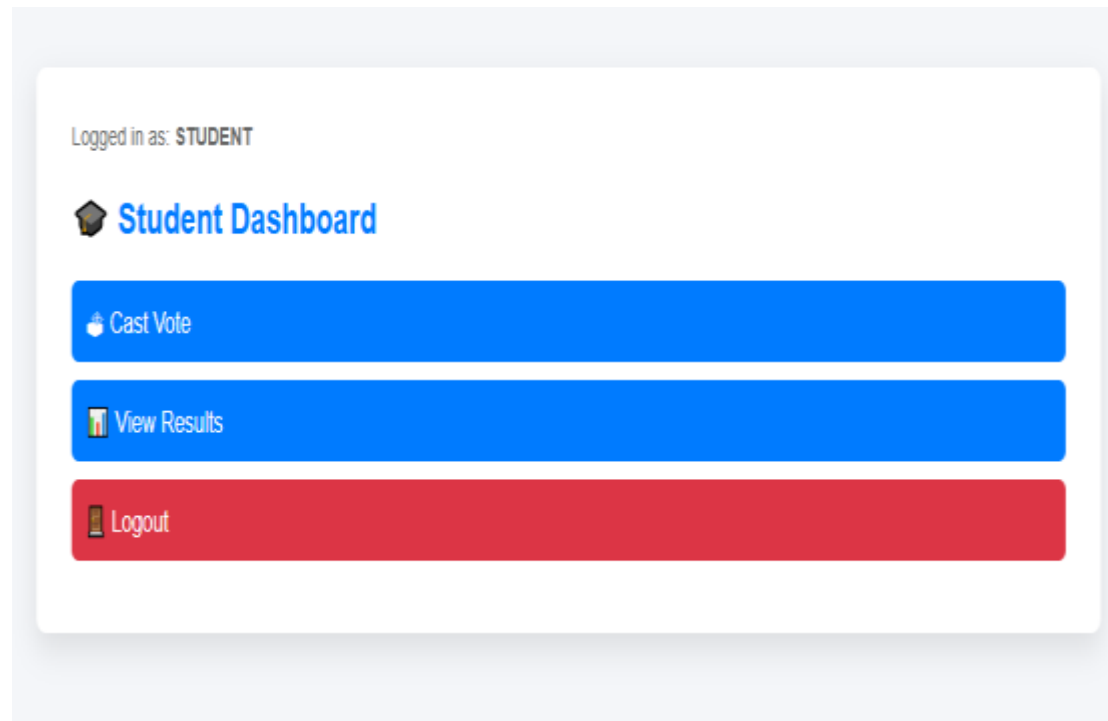


Fig 3.2.5.1: Student Interface

In this page, the students get to cast their votes and view results and log out.

3.2.5.2 Voting Interface

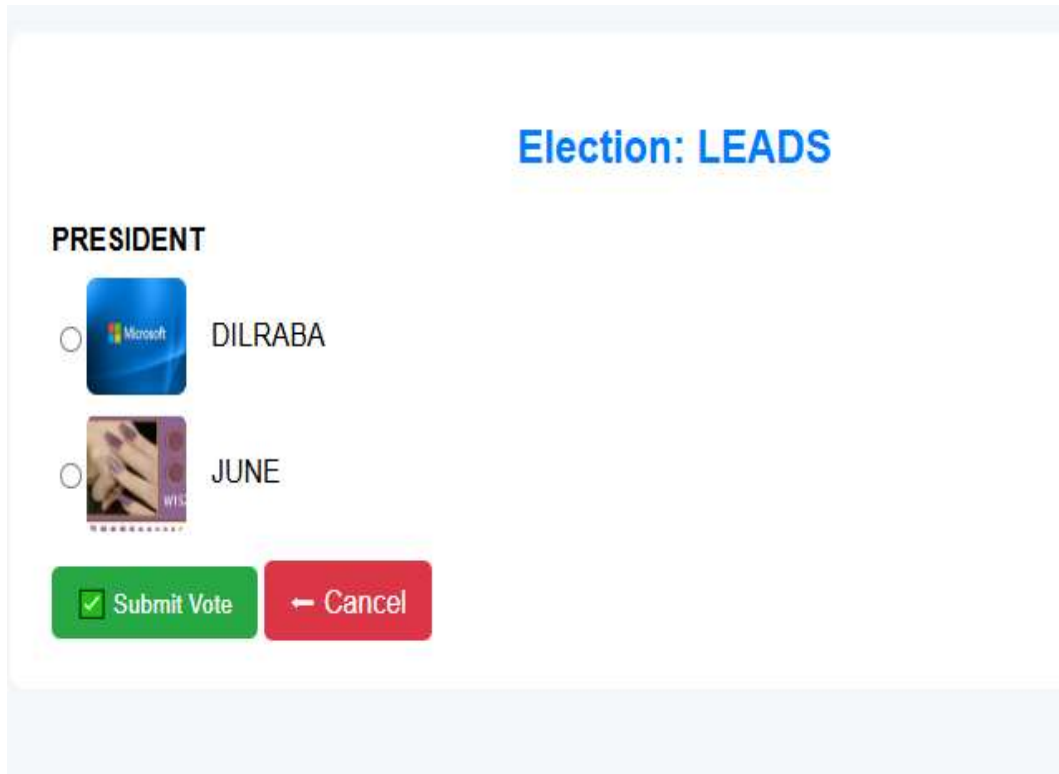


Fig 3.2.5.2: Casting Vote Interface

Allows the student to cast a vote and submit or cancel if not sure,

3.3 Logic Development

3.3.1 Student Registration Validation Code

```
register.php <
register.php > ...
1 <?php
2 require 'config/db.php';
3
4 $message = "";
5
6 if ($_SERVER["REQUEST_METHOD"] == "POST") {
7
8     $admission = $_POST['admission'];
9     $name = $_POST['name'];
10    $email = $_POST['email'];
11    $allowedDomain = "@zitech.ac.ke";
12
13    if (substr($email, -strlen($allowedDomain)) != $allowedDomain) {
14        $message = "Only Zitech University emails (@zitech.ac.ke) are allowed!";
15    }
16
17    $campus = $_POST['campus'];
18    $password = $_POST['password'];
19    $confirm = $_POST['confirm'];
20
21    // Stop if email invalid
22    if ($message) {
23    }
24
25    // Check password match
26    elseif ($password != $confirm) {
27        $message = "Passwords do not match!";
28    }
29
30    else {
31
32        // Check duplicate admission number
33        $check = mysqli_query($conn, "SELECT * FROM students WHERE admission_number='$admission'");
34
35        if (mysqli_num_rows($check) > 0) {
36            $message = "Admission number already registered!";
37        } else {
38
39            // Check duplicate admission number
40            $check = mysqli_query($conn, "SELECT * FROM students WHERE admission_number='$admission'");
41
42            if (mysqli_num_rows($check) > 0) {
43                $message = "Admission number already registered!";
44            } else {
45
46                // Hash password
47                $hashed = password_hash($password, PASSWORD_DEFAULT);
48
49                // Insert student
50                $sql = "INSERT INTO students (admission_number, full_name, email, campus, password)
51                    VALUES ('$admission', '$name', '$email', '$campus', '$hashed')";
52
53                if (mysqli_query($conn, $sql)) {
54                    header("Location: login.php");
55                    exit();
56                } else {
57                    $message = "Registration failed!";
58                }
59            }
60        }
61    }
62 }
63 ?>
```

Fig 3.3.1 Student Registration Code
Shows the PHP backend part of the page

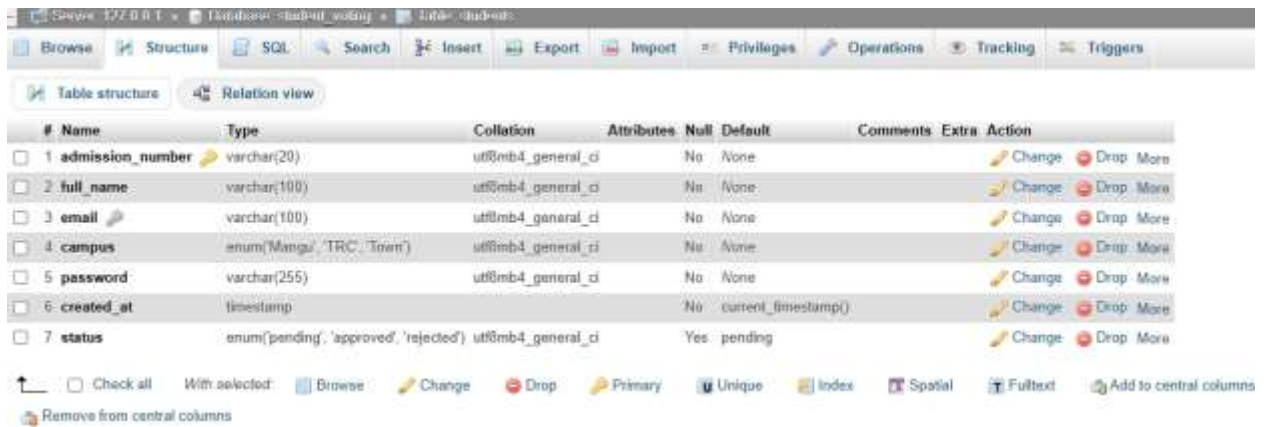
3.3.2 Student Validation Login Code

```
login.php X
login.php > ...
1  <?php
2  session_start();
3  require 'config/db.php';
4
5  $error = "";
6
7  if ($_SERVER["REQUEST_METHOD"] == "POST") {
8      $admission = trim($_POST['admission_number']);
9      $email = trim($_POST['email']); // add email field
10     $password = $_POST['password'];
11
12     // Prepared statement to select user by admission + email
13     $stmt = $conn->prepare("SELECT * FROM students WHERE admission_number = ? AND email = ?");
14     $stmt->bind_param("ss", $admission, $email);
15     $stmt->execute();
16     $result = $stmt->get_result();
17
18     if ($user = $result->fetch_assoc()) {
19         if (password_verify($password, $user['password'])) {
20             $_SESSION['user_id'] = $user['admission_number'];
21             $_SESSION['role'] = 'student';
22             $_SESSION['name'] = $user['full_name']; // store name for welcome message
23
24             header("Location: dashboard.php");
25             exit();
26         } else {
27             $error = "Incorrect password";
28         }
29     } else {
30         $error = "Student not found with this admission number and email!";
31     }
32 }
33
34
35 ?>
```

Fig 3.3.2 Student Validation Login Code

Shows the PHP code of the login page.

3.3.3 Student Database



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 admission_number	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
☐	2 full_name	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 email	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 campus	enum('Manga', 'TRC', 'Town')	utf8mb4_general_ci		No	None			Change Drop More
☐	5 password	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
☐	6 created_at	timestamp			No	current_timestamp()			Change Drop More
☐	7 status	enum('pending', 'approved', 'rejected')	utf8mb4_general_ci		Yes	pending			Change Drop More

☐ Check all With selected: Browse Change Drop Primary Unique Index Spatial Fulltext Add to central columns
 Remove from central columns

Fig 3.3.3 Student Database
Shows where the student credentials are stored.

3.3.4 Admin Validation Login Code

```
admin > login.php > ...
1  <?php
2  session_start();
3
4  $error = "";
5
6  // Fixed admin credentials
7  $admin_user = "Admin001";
8  $admin_pass = "admin123";
9
10 if ($_SERVER["REQUEST_METHOD"] == "POST") {
11     $username = $_POST['username'];
12     $password = $_POST['password'];
13
14     if ($username === $admin_user && $password === $admin_pass) {
15         $_SESSION['user_id'] = $username;
16         $_SESSION['role'] = "admin";
17
18         // File is inside /admin/, dashboard.php is one level up
19         header("Location: ../dashboard.php");
20         exit();
21     } else {
22         $error = "✗ Invalid admin credentials";
23     }
24 }
25 ?>
```

Fig 3.3.4 Admin Validation Login Code
Shows the PHP part of the admin page.

3.3.5 Admin Database



Fig 3.3.5 Admin Database
shows where the admin credentials are stored.

3.3.6 Dashboard Validation Code

```
dashboard.php X
dashboard.php > html
1  <?php
2  session_start();
3  require 'config/db.php';
4
5  // Block access
6  if (!isset($_SESSION['role'])) {
7      header("Location: index.php");
8      exit();
9  }
10
11 $role = $_SESSION['role'];
12 $current = date("Y-m-d H:i:s");
13
14 /* ===== GET LATEST ELECTION ===== */
15
16 $selectionQuery = $conn->query("
17     SELECT * FROM elections
18     ORDER BY end_date DESC
19     LIMIT 1
20 ");
21
22 $selection = $selectionQuery->fetch_assoc();
23 $selection_id = $selection ? $selection['id'] : null;
24
25 /* ===== STUDENT CHECK ===== */
26
27 $alreadyVoted = false;
28
29 if ($role === "student" && $selection_id) {
30
31     $student_id = $_SESSION['user_id'];
32
33     // Check approval
34     $statusQuery = $conn->query("
35         SELECT status FROM students
36         WHERE admission_number='$student_id'
37     ");
```

```

dashboard.php > html
39     $statusRow = $statusQuery->fetch_assoc();
40
41     if ($statusRow['status'] !== 'approved') {
42         echo "<div style='text-align:center;margin-top:50px;'>
43             <h3>⚠ Your account is awaiting admin approval.</h3>
44             <a href='logout.php'>Logout</a>
45         </div>";
46         exit();
47     }
48
49     // Check if already voted
50     $voteCheck = $conn->query("
51         SELECT id FROM votes
52         WHERE student_id='$student_id'
53         AND election_id='$election_id'
54     ");
55
56     if ($voteCheck->num_rows > 0) {
57         $alreadyVoted = true;
58     }
59 }
60
61 /* ===== ADMIN STATS ===== */
62
63 $totalVotes = 0;
64 $totalStudents = 0;
65 $turnout = 0;
66
67 if ($role === "admin") {
68     $votesQuery = $conn->query("
69         SELECT COUNT(DISTINCT student_id) as total
70         FROM votes
71     ");
72     $totalVotes = $votesQuery->fetch_assoc()['total'];
73
74     $studentsQuery = $conn->query("SELECT COUNT(*) as total FROM students");
75     $totalStudents = $studentsQuery->fetch_assoc()['total'];
76
77
78     if ($totalStudents > 0) {
79         $turnout = round(($totalVotes / $totalStudents) * 100);
80     }
81 }
82 ?>
83

```

Fig 3.3.6 Dashboard Validation Code
Shows the backend of the dashboard page.

3.3.7 Manage Candidate Validation Code

```
manage_candidates.php X
admin > manage_candidates.php > ...
1  <?php
2  // Enable error reporting
3  ini_set('display_errors', 1);
4  ini_set('display_startup_errors', 1);
5  error_reporting(E_ALL);
6
7  session_start();
8  require '../config/db.php';
9
10 // Protect page: Only admin
11 if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
12     header("Location: login.php");
13     exit();
14 }
15
16 $message = "";
17
18 // Handle Add Candidate
19 if (isset($_POST['add_candidate'])) {
20     $name = trim($_POST['name']);
21     $position = trim($_POST['position']);
22     $selection_id = intval($_POST['election_id']);
23
24     if (!empty($name) && !empty($position) && $selection_id > 0) {
25
26         // Handle photo upload
27         if (isset($_FILES['photo']) && $_FILES['photo']['error'] === 0) {
28             $uploadDir = '../uploads/';
29             if (!is_dir($uploadDir)) mkdir($uploadDir, 0755);
30
31             // Clean filename
32             $filename = time() . '_' . preg_replace("/[^a-zA-Z0-9._-]/", "", basename($_FILES['photo']['name']));
33             $targetFile = $uploadDir . $filename;
34
35             $fileType = strtolower(pathinfo($targetFile, PATHINFO_EXTENSION));
36             $allowedTypes = ['jpg', 'jpeg', 'png', 'gif'];
```

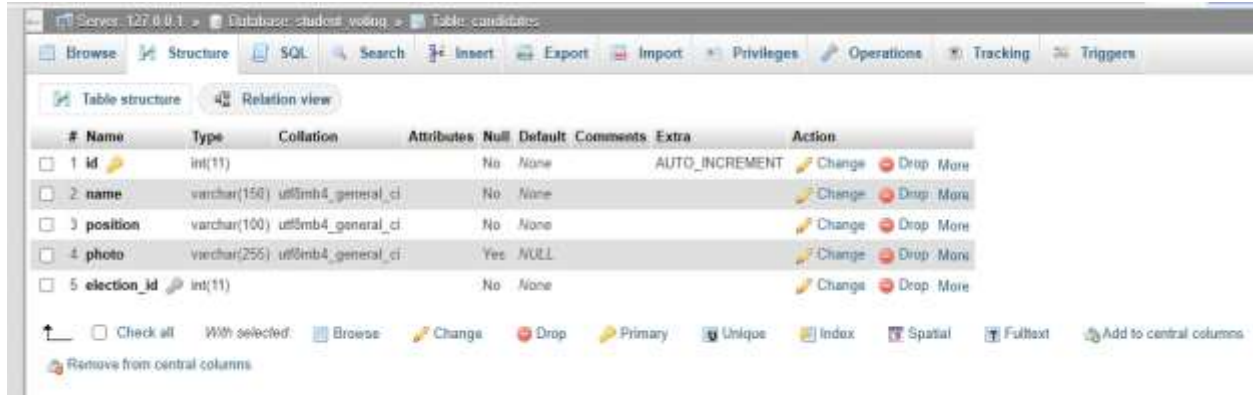
```

manage_candidates.php X
admin > manage_candidates.php > ...
37
38     if (!in_array($fileType, $allowedTypes)) {
39         $message = "✗ Only JPG, JPEG, PNG, GIF files allowed";
40     } elseif (move_uploaded_file($_FILES['photo']['tmp_name'], $targetFile)) {
41         $stmt = $conn->prepare("INSERT INTO candidates (name, position, photo, election_id) VALUES (?, ?, ?, ?)");
42         $stmt->bind_param("sssi", $name, $position, $filename, $election_id);
43
44         if ($stmt->execute()) {
45             $message = "✅ Candidate added successfully!";
46         } else {
47             $message = "✗ DB error: " . $stmt->error;
48         }
49
50         $stmt->close();
51     } else {
52         $message = "✗ Failed to upload photo";
53     }
54 } else {
55     $message = "✗ Please select a photo";
56 }
57 } else {
58     $message = "✗ Name, Position, and Election are required";
59 }
60 }
61
62 // Handle Delete Candidate
63 if (isset($_GET['delete'])) {
64     $id = intval($_GET['delete']);
65
66     // Delete photo if exists
67     $stmt = $conn->prepare("SELECT photo FROM candidates WHERE id = ?");
68     $stmt->bind_param("i", $id);
69     $stmt->execute();
70     $result = $stmt->get_result();
71     if ($row = $result->fetch_assoc()) {
72         if ($row['photo'] && file_exists("../uploads/" . $row['photo'])) {
73             unlink("../uploads/" . $row['photo']);
74         }
75     }
76 }
77
78 // Delete candidate
79 $stmt = $conn->prepare("DELETE FROM candidates WHERE id = ?");
80 $stmt->bind_param("i", $id);
81 $stmt->execute();
82 $stmt->close();
83
84 $message = "✅ Candidate deleted successfully";
85 }
86
87 // Fetch all candidates with election title
88 $candidates = $conn->query("
89     SELECT c.*, e.title AS election_title
90     FROM candidates c
91     LEFT JOIN elections e ON c.election_id = e.id
92     ORDER BY c.election_id DESC, c.position, c.name
93 ");
94
95 // Fetch all elections for dropdown
96 $elections = $conn->query("SELECT * FROM elections ORDER BY start_date DESC");
97 ?>

```

Fig 3.3.7 Manage Candidate Validation Code
shows the PHP of the manage candidates' page.

3.3.8 Candidate Database



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	name	varchar(150)	utf8mb4_general_ci		No	None			Change Drop More
☐ 3	position	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐ 4	photo	varchar(255)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐ 5	election_id	int(11)			No	None			Change Drop More

☐ Check all With selected: Browse Change Drop Primary Unique Index Spatial Fulltext Add to central columns

Remove from central columns

Fig 3.3.8 Candidate Database

Shows where the candidates' credentials are stored.

3.3.8 Monitor Votes Validation Code

```
monitor_votes.php X
admin > monitor_votes.php > ...
1  <?php
2  session_start();
3  require '../config/db.php';
4
5  // Protect page: Only admin
6  if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
7      header("Location: login.php");
8      exit();
9  }
10
11 // Fetch candidates with vote counts
12 $sql = "
13 SELECT c.id, c.name, c.position, c.photo, COUNT(v.id) AS votes
14 FROM candidates c
15 LEFT JOIN votes v ON c.id = v.candidate_id
16 GROUP BY c.id
17 ORDER BY c.position, votes DESC
18 ";
19 $result = $conn->query($sql);
20
21 // Get total votes
22 $totalVotes = $conn->query("SELECT COUNT(*) AS total FROM votes")->fetch_assoc()['total'];
23
24 ?>
25
```

Fig 3.3.8 Monitor Votes Validation Code
shows the PHP of the monitor votes page.

3.3.9 View Results Validation Code

```
results.php > ...
1  <?php
2  session_start();
3  require 'config/db.php';
4
5  // Protect page
6  if (!isset($_SESSION['role'])) {
7      header("Location: index.php");
8      exit();
9  }
10
11  $role = $_SESSION['role'];
12  $current_time = date("Y-m-d H:i:s");
13
14  // Get latest ACTIVE election (real-time)
15  $selectionQuery = mysqli_query($conn, "
16      SELECT * FROM elections
17      WHERE start_date <= '$current_time'
18      AND end_date >= '$current_time'
19      ORDER BY start_date DESC
20      LIMIT 1
21  ");
22
23  $selection = mysqli_fetch_assoc($selectionQuery);
24
25  $message = "";
26
27  if (!$selection) {
28      $message = "No active election at the moment.";
29  }
30
```

```

results.php > ...
30
31 // Fetch results
32 $results = [];
33 $totalVotes = 0;
34
35 if ($selection) {
36     $candQuery = mysqli_query($conn, "
37         SELECT c.*, COUNT(v.id) AS votes
38         FROM candidates c
39         LEFT JOIN votes v
40         ON c.id = v.candidate_id
41         AND v.election_id='".$selection['id']."'
42         WHERE c.election_id='".$selection['id']."'
43         GROUP BY c.id
44         ORDER BY c.position, votes DESC
45     ");
46
47     while ($row = mysqli_fetch_assoc($candQuery)) {
48         $results[$row['position']][] = $row;
49         $totalVotes += $row['votes'];
50     }
51 }
52 ?>

```

3.3.9 View Results Validation Code
shows the backend of the view results page

3.3.10 Publish Results Validation Code

```
admin > publish_results.php > ...
1  <?php
2  session_start();
3  require '../config/db.php';
4
5  // ===== PROTECT PAGE =====
6  if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
7      header("Location: ../login.php");
8      exit();
9  }
10
11 // ===== VALIDATE ID =====
12 if (!isset($_GET['id']) || empty($_GET['id'])) {
13     header("Location: ../dashboard.php?error=no_id");
14     exit();
15 }
16
17 $id = intval($_GET['id']);
18
19 // ===== CHECK IF ELECTION EXISTS =====
20 $checkExists = $conn->prepare("SELECT id, end_date, results_published FROM elections WHERE id = ?");
21 $checkExists->bind_param("i", $id);
22 $checkExists->execute();
23 $result = $checkExists->get_result();
24
25 if ($result->num_rows === 0) {
26     header("Location: ../dashboard.php?error=not_found");
27     exit();
28 }
29
```

```
publish_results.php X
admin > publish_results.php > ...

30 $selection = $result->fetch_assoc();
31
32 // ===== CHECK IF ELECTION HAS ENDED =====
33 if ($selection['end_date'] > date("Y-m-d H:i:s")) {
34     header("Location: ../dashboard.php?error=not_ended");
35     exit();
36 }
37
38 // ===== CHECK IF ALREADY PUBLISHED =====
39 if ($selection['results_published'] == 1) {
40     header("Location: ../dashboard.php?error=already_published");
41     exit();
42 }
43
44 // ===== UPDATE RESULTS =====
45 $stmt = $conn->prepare("
46     UPDATE elections
47     SET results_published = 1
48     WHERE id = ?
49 ");
50
51 $stmt->bind_param("i", $id);
52
53 if ($stmt->execute()) {
54
55     if ($stmt->affected_rows > 0) {
56         header("Location: ../dashboard.php?published=1");
57         exit();
58     } else {
59         header("Location: ../dashboard.php?error=failed");
60         exit();
61     }
62
63 } else {
64     header("Location: ../dashboard.php?error=failed");
65     exit();
66 }
67 ...>
```

Fig 3.3.10 Publish Results Validation Code
shows the PHP f the publish results page.

3.3.11 Election Database



The screenshot shows a database management interface with a toolbar at the top containing buttons for Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, Tracking, and Triggers. Below the toolbar, there are two tabs: 'Table structure' and 'Relation view'. The 'Table structure' tab is active, displaying a table with 6 columns: #, Name, Type, Collation, Attributes, Null, Default, Comments, Extra, and Action. The table lists the following fields:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	title	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
3	start_date	datetime			No	None			Change Drop More
4	end_date	datetime			No	None			Change Drop More
5	created_at	timestamp			No	current_timestamp()			Change Drop More
6	results_published	tinyint(1)			Yes	0			Change Drop More

Fig 3.3.11 Election Database
shows the storage of the election data.

3.3.12 Vote Validation Code

```
vote.php > ...
1  <?php
2  session_start();
3  require 'config/db.php';
4
5  // Protect page: only students
6  if (!isset($_SESSION['role']) || $_SESSION['role'] !== 'student') {
7      header("Location: login.php");
8      exit();
9  }
10
11  $student_id = $_SESSION['user_id'];
12  $message = "";
13
14  // Fetch active election
15  $current_time = date("Y-m-d H:i:s");
16  $selectionQuery = mysqli_query($conn, "
17      SELECT * FROM elections
18      WHERE start_date <= '$current_time'
19      AND end_date >= '$current_time'
20      ORDER BY end_date ASC
21      LIMIT 1
22  ");
23  $selection = mysqli_fetch_assoc($selectionQuery);
24
25  // If no active election
26  if (!$selection) {
27      $message = "Voting is not active right now. It may not have started or has already ended.";
28  }
29
30  // Handle vote submission
31  if ($_SERVER["REQUEST_METHOD"] == "POST" && $selection) {
32      $selection_id = $selection['id'];
33      $votes = $_POST['vote'] ?? [];
34  }
```

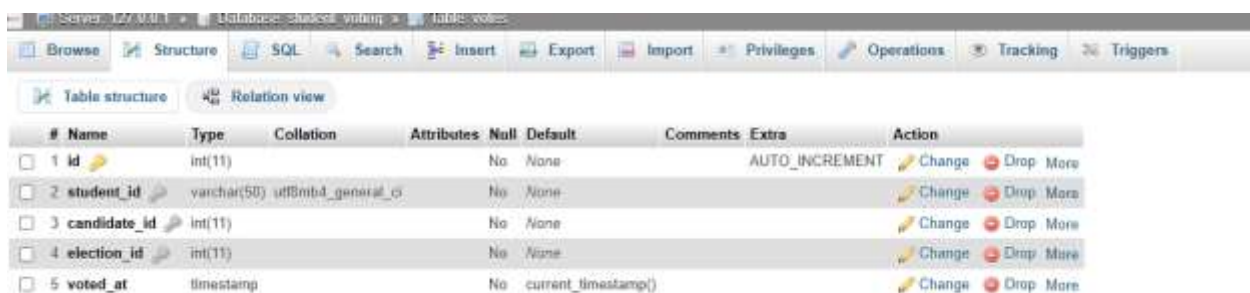
```

vote.php > ...
34
35 // Prevent double voting
36 $checkVote = mysqli_query($conn, "
37     SELECT * FROM votes
38     WHERE student_id='$student_id'
39     AND election_id='$election_id'
40 ");
41
42 if (mysqli_num_rows($checkVote) > 0) {
43     header("Location: dashboard.php?already_voted=1");
44     exit();
45 }
46
47 // Save votes
48 foreach ($votes as $position => $candidate_id) {
49     $stmt = mysqli_prepare($conn, "
50         INSERT INTO votes (student_id, candidate_id, election_id)
51         VALUES (?, ?, ?)
52     ");
53     mysqli_stmt_bind_param($stmt, "sii", $student_id, $candidate_id, $election_id);
54     mysqli_stmt_execute($stmt);
55     mysqli_stmt_close($stmt);
56 }
57
58 header("Location: dashboard.php?success=1");
59 exit();
60 }
61 ?>
62

```

Fig 3.3.12 Vote Validation Code
shows the PHP code of the vote page

3.3.13 Vote Database



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	student_id	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
3	candidate_id	int(11)			No	None			Change Drop More
4	election_id	int(11)			No	None			Change Drop More
5	voted_at	timestamp			No	current_timestamp()			Change Drop More

Fig 3.3.13 Vote Database
shows the where the votes cast are stored.

3.4 Testing

For testing, each module of the system was tested individually first and then together to ensure proper interaction of the system. First, it was the user authentication the login and registration part where I tested using valid and invalid credentials, verified that only registered students with the valid school email could access the system. It also checks session handling after login and logout and also improve password verification by ensuring they were hashed.

The student management module was tested by performing create, read, update, and delete (CRUD) operations. Data entered through forms was verified against database records to ensure consistency. Errors related to incorrect SQL queries were identified and corrected by ensuring proper column naming and query structure.

For the election management module, different elections were created with varying start and end dates. The system was tested to confirm that only active elections were displayed to users. Issues related to incorrect date comparisons were identified and corrected by refining the logic used to filter elections based on the current time.

The voting functionality was tested by submitting votes under different scenarios. It was verified that a user could only vote once per election. Initially, duplicate voting was possible due to insufficient validation, but this was corrected by enforcing constraints at both the application and database levels. Redirect mechanisms were also added after voting to prevent duplicate submissions when refreshing the page.

The results display module was tested to ensure that vote counts were correctly calculated and displayed. Errors in SQL queries were identified and fixed, ensuring accurate tallying of votes. Sorting mechanisms were also implemented to display candidates based on the number of votes received.

Role-based access control was tested by attempting to access restricted pages using different user roles. Unauthorized access was successfully prevented by implementing session-based role checks across all protected pages. Users without proper authorization were redirected appropriately.

Finally, form validation and database connectivity were tested extensively. Both client-side and server-side validation techniques were implemented to ensure that only valid data was processed. Database connection issues were also identified and resolved by correcting configuration settings and handling connection errors correctly.

Overall, the testing process helped identify and resolve various issues, improving the reliability, security, and functionality of the system.

3.5 Deployment

The developed student leaders online voting system was deployed as a web-based application to allow access by users from any location using a web browser. The system was hosted on InfinityFree, a free web hosting platform that supports PHP and MySQL databases.

To deploy the system, the project files were first prepared locally using a development environment. All PHP scripts, HTML files, and related resources were organized into a structured directory. The database was created using MySQL, and all required tables such as students, elections, candidates, and votes were exported from the local environment.

The deployment process began by creating an account on InfinityFree and setting up a hosting space. A new MySQL database was created through the hosting control panel, and the exported database file was imported using the provided database management tool (phpMyAdmin). The database configuration file in the project (db.php) was then updated with the correct database name, username, password, and host details provided by the hosting platform.

After configuring the database, all system files were uploaded to the hosting server using the File Manager provided by InfinityFree. The files were placed in the appropriate public directory (htdocs) to make them accessible via the web. Once the upload was complete, the system was tested online to ensure that all functionalities, including login, voting, and results display, were working correctly in the live environment.

During deployment, some issues such as database connection errors were encountered due to incorrect credentials. These were resolved by updating the configuration file with accurate connection details. File path issues were also corrected to ensure proper linking between pages.

The system is now accessible online through the following link:

<https://univote.infinityfreeapp.com/index.php>

This deployment process transformed the system from a locally developed application into a fully functional web-based platform that can be accessed and used by students and administrators from any device with internet connectivity.

CHAPTER FOUR: CONCLUSION AND RECOMMENDATION

4.1 Conclusion

This project has presented the design and development of a student voting system aimed at improving the efficiency, transparency, and accessibility of student elections. Chapter One introduced the background of the study, highlighting the limitations of traditional voting methods and outlining the objectives of the system. Chapter Two reviewed existing literature and similar systems, identifying gaps that informed the development of a more secure and user-friendly solution. Chapter Three detailed the system design and implementation, including the technologies used, database structure, system logic, and user interfaces.

Throughout the development process, several valuable lessons were learned, particularly in system design, database management, and integrating both frontend and backend components to create a functional application. The project also enhanced practical skills in debugging, testing, and deploying a web-based system. However, challenges were encountered, such as handling database connection errors, ensuring system security, and managing real-time election conditions like vote validation and result accuracy. Despite these challenges, the project was successfully completed, providing a reliable solution that can be further improved and scaled in the future.

4.2 Recommendation

If you choose to further improve this Student Voting System, there are several enhancements that can be implemented to make the system more secure, user-friendly, and efficient.

First, the login and authentication process can be strengthened by introducing an email verification mechanism. Students should be required to verify their institutional email addresses through a One-Time Password (OTP) sent to their email. The system can generate these OTPs dynamically, with the admin having control over their configuration and validity period. This would reduce unauthorized access and improve system security. Additionally, a password recovery feature should be implemented to allow users to request a password reset in case they forget their credentials. This can be done through a secure email-based reset link or OTP verification process.

Secondly, the user interface can be enhanced by personalizing the student dashboard. For example, displaying a greeting such as “Hello, [Student Name or Admission Number]” at the top of the dashboard would improve user experience and make the system more interactive.

In addition, incorporating graphical representations such as bar charts would improve the visualization of election results. This would allow both students and administrators to quickly interpret voting outcomes and trends instead of relying only on numerical data.

Another important improvement would be the integration of notification systems. Students can receive reminders via email or SMS about upcoming elections, voting deadlines, and result announcements. This would help increase participation and ensure students stay informed.

Furthermore, the system can be enhanced by allowing votes to be categorized based on departments. This would enable quicker and more meaningful analysis of voting patterns across different groups within the institution.

An AI-powered chatbot can also be integrated into the system to assist users. This chatbot would help answer common questions such as how to vote, election timelines, and login issues. It would provide real-time support, reduce the workload on administrators, and improve overall usability.

To further strengthen the integrity of the system, additional measures can be implemented to prevent multiple voting. These may include tracking IP addresses, using device fingerprinting, or implementing stricter session controls to ensure that each student votes only once.

Accessibility improvements should also be considered to make the system usable by a wider range of students, including those with disabilities. This can include support for screen readers, improved color contrast, and keyboard navigation.

Lastly, implementing a backup and recovery system is highly recommended. Automated database backups and restore mechanisms would ensure that election data is not lost in case of system failure, enhancing reliability and data security.

Overall, these enhancements build on the existing working system and aim to improve security, usability, and scalability through the use of more advanced technologies.

REFERENCES

- Adrian Petitpas, Julien M (2021, May 28). *“Does E-Voting matter for turnout, and to whom?”*: ScienceDirect. Retrieved from <https://www.sciencedirect.com/science/article/pii/S0261379420301244>
- Aristotle, Technology News (2023, October 7). *“Voting Software Vulnerabilities: Concerns and Risks”*
Aristotle Now you Know. Retrieved from <https://www.aristotle.com/blog/2023/10/voting-software-vulnerabilities-concerns-and-risks>
- Classter (2024). *“Creating a Seamless User Experience in Your Student Information System”*: Classter. Retrieved from <https://www.classter.com/blog/edtech/student-information-systems/creating-a-seamless-user-experience-in-your-student-information-system/>
- DevX (2025 April, 20). *“12 Best Practices and Approaches for System Backups and Disaster Recovery”*: DevX. Retrieved from <https://www.devx.com/systems-administration/12-best-practices-and-approaches-for-system-backups-and-disaster-recovery/>
- ElectionBuddy (2025). *“The Advantages and Disadvantages of Online Voting Systems”*: ElectionBuddy. Retrieved from <https://electionbuddy.com/blog/2022/04/20/the-advantages-and-disadvantages-of-online-voting-systems/>
- GeeksforGeeks (2025). *“Essential Security Measures in System Design”*: GeeksforGeeks. Retrieved from <https://www.geeksforgeeks.org/system-design/essential-security-measures-in-system-design/>
- IriaAlumni (2023, July 26). *“The security of electronic voting: vulnerabilities and solutions.”*: Inria Retrieved from <https://www.inria.fr/en/security-electronic-voting-digital-security-confidentiality>
- Jotform (n.d). *“Online Voting Tool”*: Jotform. Retrieved from <https://www.jotform.com/online-voting-tool/>
- Nandasaba Wilson (n.d.). *“Project Report: Online Voting System”*: SlideShare. Retrieved from <https://www.slideshare.net/slideshow/project-reportonline-voting-system/13784947>
- Ratibor Sekirov (2021, November 3). *“Benefits of Web-Based management System”*: Aspurity. Retrieved from <https://aspurity.com/blog/webbased-management-system>

Ryan N. (2022, April 20). *“Advantages and Disadvantages of Online Voting System”*: ElectionBuddy. Retrieved from <https://electionbuddy.com/blog/2022/04/20/the-advantages-and-disadvantages-of-online-voting-systems/>

Scribd (2023). *“Online Voting System Documentation”*: Scribd. Retrieved from <https://www.scribd.com/document/730407965/Online-Voting-System-Documentation>

TutorialsPoint (n.d.). *“PHP Tutorial”*: TutorialsPoint. Retrieved from <https://www.tutorialspoint.com/php/index.htm>

Uzok, C., Cadet, & Ojukwu (2024). *“Leveraging AI-Powered Chatbots to Enhance Customer Service Efficiency and Future Opportunities in Automated Support”*: Computer Science & IT Research Journal. Retrieved from https://www.researchgate.net/publication/385230161_Leveraging_AI-Powered_chatbots_to_enhance_customer_service_efficiency_and_future_opportunities_in_automated_support

Verifalia (2025). *“Validate Email Address”*: Verifalia. Retrieved from <https://verifalia.com/validate-email>

W3Schools (n.d.). *“PHP Tutorial”*: W3Schools. Retrieved from <https://www.w3schools.com/php/>

W3Schools (n.d.). *“HTML CSS”*: W3Schools. Retrieved from; q https://www.w3schools.com/html/html_css.asp

Zoran Cocoara (2023, November 3). *“Encryption: Protecting sensitive information in digital age”*:

Endpoint Protector. Retrieved from <https://www.endpointprotector.com/blog/data-encryption-protecting-sensitive-information/>